

LAPORAN PRAKTIKUM
STRUKTUR DATA
IMPLEMENTASI *ALGORITMA SORTING*
MENGGUNAKAN BAHASA *JAVA*



Disusun Oleh:

Haliya Isma Husna Putri Ahmadi
2511532002

Dosen Pengampu:
Wahyudi. Dr. S.T.M.T

Asisten Praktikum:
Muhammad Galid

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji dan Syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas Rahmat dan karunia-Nya, penulis dapat menyelesaikan laporan praktikum Struktur Data pekan 8 dengan judul “Implementasi Algoritma Sorting Menggunakan Bahasa *Java*” tepat pada waktunya.

Penyusunan laporan ini bertujuan untuk memenuhi tugas mata kuliah Praktikum Struktur Data, sekaligus sebagai sarana pembelajaran bagi penulis dalam memahami konsep algoritma sorting serta penerapannya dalam pemrograman *Java*. Melalui praktikum ini, penulis mempelajari bagaimana proses pengurutan data dilakukan menggunakan beberapa metode *sorting*, yaitu *Shell Sort*, *Quick Sort*, dan *Merge Sort*.

Selain itu penulis juga mengimplementasikan visualisasi proses pengurutan data menggunakan *Java GUI* sehingga proses sorting dapat diamati langkah demi langkah. Dengan implementasi tersebut, penulis dapat memahami cara kerja masing-masing algoritma sorting, mulai dari proses pertukaran data, pencarian nilai minimum, hingga penyisipan elemen pada posisi yang sesuai. Praktikum ini juga membantu penulis memahami perbedaan mekanisme dan efisiensi dari setiap metode pengurutan data.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, penulis sangat terbuka terhadap kritik dan saran yang membangun demi perbaikan di masa yang akan datang. Ucapan terima kasih yang sebesar-besarnya penulis sampaikan kepada dosen pengampu, asisten praktikum, serta semua pihak yang terlibat dalam pelaksanaan praktikum dan penyusunan laporan ini. Semoga laporan ini dapat memberikan manfaat dan menambah wawasan bagi penulis maupun pembaca.

Padang, 03 June 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	ii
DAFTAR ISI.....	iii
BAB I PENDAHULUAN.....	5
1.1 Latar Belakang.....	5
1.2 Tujuan	6
1.3 Manfaat	6
BAB II PEMBAHASAN.....	7
2.1 Dasar Teori.....	7
2.1.1 <i>BubbleSort</i>	7
2.1.2 <i>ShellSort</i>	7
2.1.3 <i>QuickSort</i>	8
2.1.4 <i>MergeSort</i>	9
2.1.5 Visualisasi Program <i>Sorting</i> dengan <i>Java GUI</i>	9
2.2 Langkah Kerja	10
2.2.1 Pembuatan <i>Package</i> dan <i>Class</i>	10
2.2.2 Membuat Program <i>ShellSort_251153200</i>	14
2.2.3 Membuat Program <i>QuickSort_2511532002</i>	16
2.2.4 Membuat Program <i>MergeSort_2511532002</i>	20
2.2.5 Membuat Program <i>BubleSortGUI_2511532002</i>	25
2.2.6 Membuat Program <i>MergeSortGUI_2511532002</i>	32
2.2.7 <i>Upload</i> ke GitHub	42
2.3 Hasil.....	45
2.3.1 Hasil <i>ShellSort_2511532002</i>	45
2.3.2 Hasil <i>QuickSort_2511532002</i>	45

2.3.3	Hasil <i>MergeSort</i> _2511532002	46
2.3.4	Hasil <i>BubleSortGUI</i> _2511532002	46
2.3.5	Hasil <i>MergeSortGUI</i> _2511532002.....	47
BAB III KESIMPULAN.....		48
3.1	Kesimpulan	48
3.2	Saran	48
DAFTAR PUSTAKA.....		49
TUGAS PRAKTIKUM PEKAN 5.....		50
1.	Kode Program	50
1.1	Kelas <i>Lagu</i> _2511532002	50
1.2	Kelas <i>SortingNIM</i> _2511532002.....	52
2.	Hasil	60
3.	Kesimpulan	61

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi yang semakin pesat menuntut adanya kemampuan dalam mengelola data secara efisien dan terstruktur. Dalam dunia pemrograman, pengolahan data menjadi salah satu aspek penting yang harus diperhatikan agar sistem dapat bekerja dengan optimal. Salah satu bentuk pengolahan data yang paling dasar namun sangat penting adalah proses pengurutan data (*sorting*).

Sorting merupakan proses menyusun data dalam urutan tertentu, baik secara menaik (*ascending*) maupun menurun (*descending*). Proses ini berperan penting dalam berbagai aplikasi karena dapat mempermudah pencarian, analisis, dan pengelolaan data. Dalam pemrograman Java, *sorting* dapat dilakukan dengan dua cara, yaitu menggunakan algoritma *sorting* manual maupun menggunakan fungsi bawaan seperti *Arrays.sort()*.

Pada praktikum ini, penulis mempelajari dan mengimplementasikan beberapa algoritma *sorting* dasar, yaitu *Bubble Sort*, *Shell Sort*, *Quick Sort*, dan *Merge Sort*. Ketiga algoritma tersebut memiliki cara kerja yang berbeda dalam mengurutkan data, namun memiliki tujuan yang sama yaitu menghasilkan data yang terurut dengan benar. Selain itu, praktikum ini juga mengimplementasikan visualisasi proses *sorting* menggunakan *Java GUI (Swing)*.

Melalui praktikum ini, penulis diharapkan dapat memahami konsep dasar algoritma *sorting*, cara kerja masing-masing metode, serta mampu mengimplementasikannya dalam bahasa pemrograman *Java* sebagai dasar pengembangan aplikasi yang lebih kompleks di masa mendatang.

1.2 Tujuan

Tujuan dari pelaksanaan praktikum ini adalah sebagai berikut:

- 1.2.1 Memahami konsep dasar algoritma sorting sebagai metode pengurutan data dalam struktur pemrograman.
- 1.2.2 Mengetahui dan memahami cara kerja beberapa algoritma sorting, yaitu *Shell Sort*, *Quick Sort*, dan *Merge Sort* dalam bahasa pemrograman Java.
- 1.2.3 Mengimplementasikan visualisasi proses sorting menggunakan *Java GUI (Swing)* untuk memahami proses pengurutan data secara bertahap dan interaktif.
- 1.2.4 Melatih kemampuan dalam menganalisis dalam membandingkan cara kerja dan langkah-langkah setiap algoritma *sorting*.

1.3 Manfaat

Manfaat yang diharapkan dari pelaksanaan praktikum ini antara lain:

- 1.3.1 Menambah pemahaman mengenai konsep dan penerapan algoritma *sorting* dalam pemrograman *Java*.
- 1.3.2 Membantu Mahasiswa memahami proses pengurutan data menggunakan berbagai metode sorting seperti *Bubble Sort*, *Shell Sort*, *Quick Sort*, dan *Merge Sort*. Serta visualisasi algoritma menggunakan *Java GUI* sehingga proses sorting dapat diamati secara langsung.
- 1.3.3 Melatih kemampuan logika pemrograman dalam mengelola dan mengurutkan data secara sistematis.

BAB II

PEMBAHASAN

2.1 Dasar Teori

Sorting adalah proses pengurutan data berdasarkan aturan tertentu, biasanya dalam urutan menaik (*ascending*) atau menurun (*descending*). Dalam dunia pemrograman, *sorting* digunakan untuk mempermudah pencarian, pengolahan, dan analisis data.

Algoritma *sorting* merupakan langkah-langkah sistematis yang digunakan untuk mengurutkan elemen data dalam struktur seperti *array* atau *list*. Setiap algoritma memiliki cara kerja yang berbeda, namun tujuan akhirnya sama yaitu menghasilkan data yang terurut.

2.1.1 *Bubble Sort*

Bubble Sort adalah algoritma *sorting* yang bekerja dengan cara membandingkan dua elemen yang bersebelahan, kemudian menukarnya jika urutannya salah.

Proses ini dilakukan secara berulang hingga seluruh data berada dalam keadaan terurut.

Karakteristik *Bubble Sort*:

1. Sederhana dan mudah dipahami
2. Kurang efisien untuk data besar
3. Kompleksitas waktu: $O(n^2)$

2.1.2 *Shell Sort*

Shell Sort merupakan pengembangan dari *Insertion Sort* yang bekerja dengan membandingkan elemen-elemen yang dipisahkan oleh jarak tertentu (*gap*). Nilai *gap* akan terus diperkecil hingga menjadi 1, sehingga pada tahap akhir data dapat diurutkan dengan lebih cepat dibandingkan *Insertion Sort* biasa.

Karakteristik *Shell Sort*:

1. Merupakan pengembangan dari algoritma *Insertion Sort*.
2. Menggunakan konsep *gap* untuk mempercepat proses pengurutan.
3. Lebih efisien dibandingkan *Insertion Sort* pada data berukuran besar.
4. Kompleksitas waktu bergantung pada urutan *gap* yang digunakan, dengan performa yang umumnya lebih baik daripada $O(n^2)$.

2.1.3 *Quick Sort*

Quick Sort adalah algoritma sorting yang menggunakan metode *divide and conquer*. Algoritma ini bekerja dengan memilih sebuah elemen sebagai *pivot*, kemudian membagi data menjadi dua bagian, yaitu elemen yang lebih kecil dari *pivot* dan elemen yang lebih besar dari *pivot*. Proses tersebut dilakukan secara rekursif hingga seluruh data terurut.

Karakteristik *Quick Sort*:

1. Menggunakan metode *divide and conquer*.
2. Memiliki performa yang sangat baik untuk sebagian besar kasus.
3. Kompleksitas waktu rata-rata $O(n \log n)$.
4. Kompleksitas waktu terburuk $O(n^2)$ apabila pemilihan *pivot* tidak optimal.

2.1.4 Merge Sort

Merge Sort merupakan algoritma sorting yang juga menggunakan pendekatan *divide and conquer*. Algoritma ini bekerja dengan membagi *array* menjadi beberapa bagian yang lebih kecil, kemudian mengurutkan setiap bagian secara rekursif, dan akhirnya menggabungkan kembali bagian-bagian tersebut menjadi satu array yang telah terurut.

Karakteristik *Merge Sort*:

1. Menggunakan metode *divide and conquer*.
2. Memiliki performa yang stabil pada berbagai kondisi data.
3. Kompleksitas waktu $O(n \log n)$ pada kondisi terbaik, rata-rata, maupun terburuk.
4. Membutuhkan memori tambahan untuk proses penggabungan data.

2.1.5 Visualisasi Program Sorting dengan Java GUI

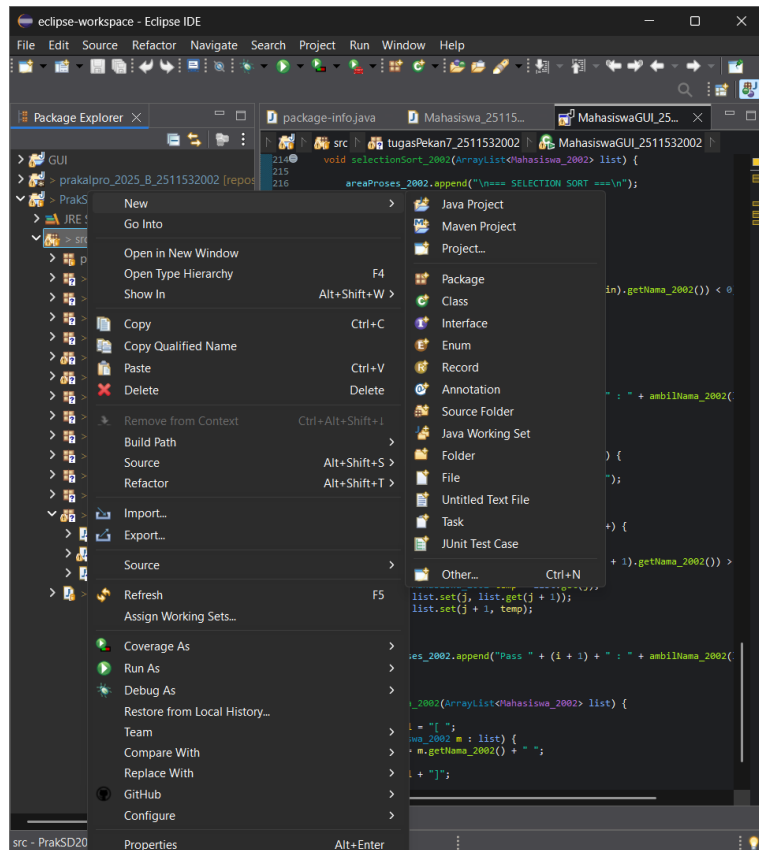
Java GUI (Graphical User Interface) merupakan antarmuka grafis yang memungkinkan pengguna berinteraksi dengan program melalui komponen visual seperti tombol, kotak teks, label, dan panel. Dalam praktikum ini, *Java Swing* digunakan untuk memvisualisasikan proses sorting secara bertahap sehingga pengguna dapat mengamati perubahan posisi data pada setiap langkah pengurutan.

Visualisasi algoritma membantu pengguna memahami proses kerja sorting secara lebih intuitif dibandingkan hanya melihat hasil akhir pengurutan. Dengan demikian, konsep algoritma dapat dipahami dengan lebih mudah dan interaktif.

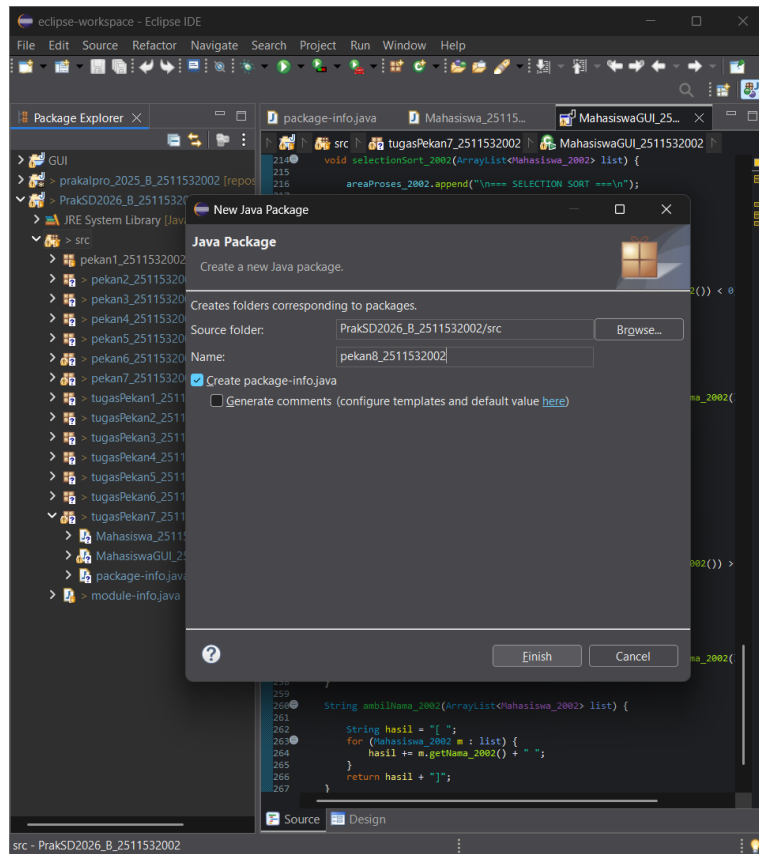
2.2 Langkah Kerja

2.2.1 Pembuatan *Package* dan *Class*

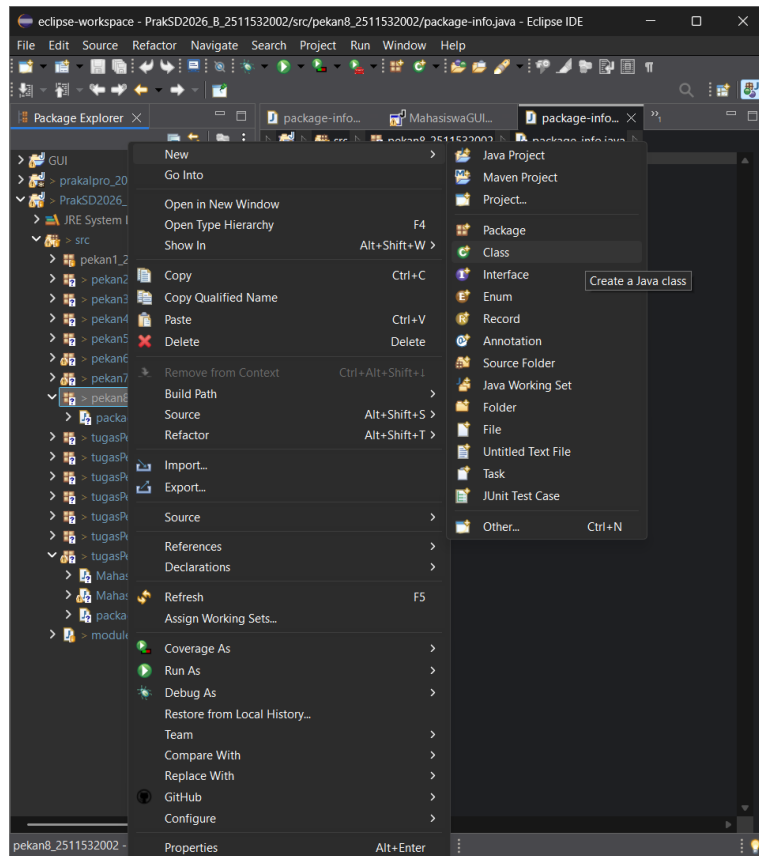
- 1) Sebelum membuat suatu program *user* perlu untuk membuat *package* terlebih dahulu. Klik kanan pada *src* pada *repository* “PrakSD2026_B_2511532002” lalu pilih “New” dan klik “Package”.



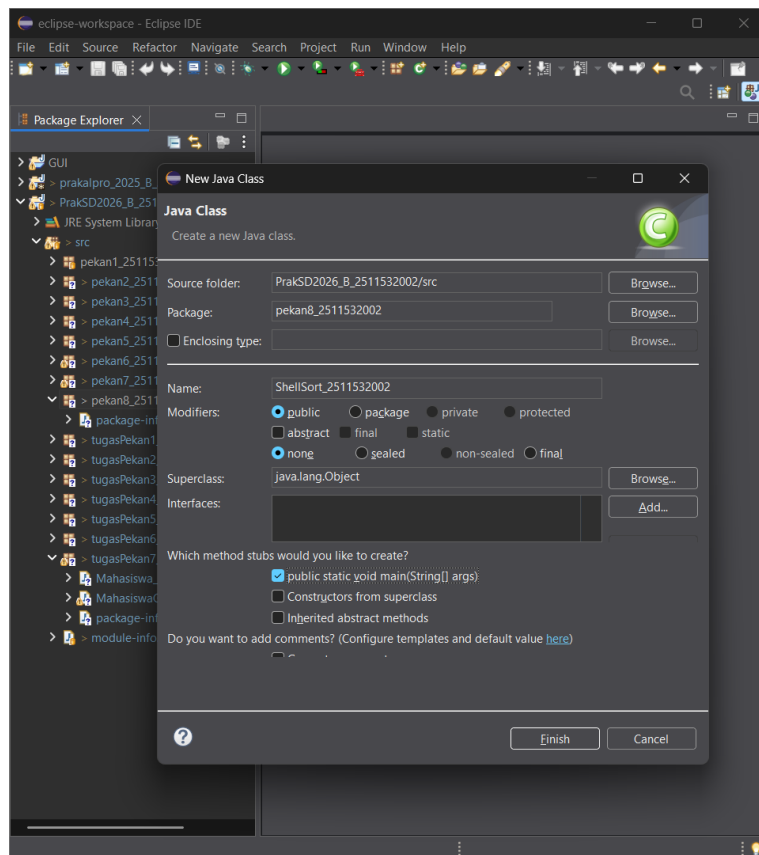
- 2) Setelah itu akan muncul *pop up* “Java Package”, buat nama *package* dengan ketentuan tanpa *capslock*, *space*, dan karakter khusus lainnya. Lalu klik “*finnish*”.



- 3) Selanjutnya adalah membuat membuat *class*. pada package yang telah dibuat tapi klik kanan lalu pilih “New” dan klik “Class”.



- 4) Setelah itu pada menu “*Java Class*” buat nama *class*, dengan ketentuan nama harus capslock diawal kalimat dan tanpa spasi.



2.2.2 Membuat Program *ShellSort_2511532002*

```
package pekan8_2511532002;

public class ShellSort_2511532002 {
    public static void shellSort_2511532002 (int[] A_2002) {
        int n_2002= A_2002.length;
        int gap_2002=n_2002/2;
        while (gap_2002 >0) {
            for (int i_2002=gap_2002; i_2002<n_2002; i_2002++) {
                int temp_2002=A_2002[i_2002];
                int j_2002=i_2002;
                while (j_2002 >= gap_2002 && A_2002[j_2002-
gap_2002]> temp_2002) {
                    A_2002[j_2002]=A_2002[j_2002-gap_2002];
                    j_2002=j_2002-gap_2002;
                }
                A_2002[j_2002]=temp_2002;
            }
            gap_2002=gap_2002/2;
        }
    }

    public static void main(String[] args) {
        int [] data_2002= {3, 10, 4, 6, 8, 9, 7, 2, 1, 5};
        System.out.print("Sebelum: ");
        printArray_2002(data_2002);

        shellSort_2511532002(data_2002);

        System.out.print("Sesudah (ShellSort): ");
        printArray_2002(data_2002);
    }

    public static void printArray_2002(int[] arr) {
        for (int i_2002:arr) System.out.print(i_2002 + " ");
        System.out.println();
    }
}
```

Setelah membuat *class ShellSort_2511532002*, maka mahasiswa dapat menulis *syntax* nya.

1. Membuat *class Shell Sort*
`public class ShellSort_2511532002 {`
 Syntax di atas digunakan untuk membuat class bernama *ShellSort_2511532002* sebagai tempat penulisan program algoritma *Shell Sort*.
2. Membuat *method Shell Sort*
`public static void shellSort_2511532002 (int[] A_2002) {`
`int n_2002= A_2002.length;`
`int gap_2002=n_2002/2;`
`while (gap_2002 >0) {`
`for (int i_2002=gap_2002; i_2002<n_2002;`
`i_2002++) {`
`int temp_2002=A_2002[i_2002];`
`int j_2002=i_2002;`

```

        while (j_2002 >= gap_2002 &&
A_2002[j_2002-gap_2002]> temp_2002) {
            A_2002[j_2002]=A_2002[j_2002-gap_2002];
            j_2002=j_2002-gap_2002;
        }
        A_2002[j_2002]=temp_2002;
    }
    gap_2002=gap_2002/2;
}
}

```

Method shellSort_2511532002() digunakan untuk melakukan proses pengurutan data menggunakan algoritma *Shell Sort*. Algoritma ini bekerja dengan membandingkan elemen-elemen yang memiliki jarak tertentu (*gap*), kemudian secara bertahap memperkecil nilai *gap* hingga bernilai 1. Perulangan *while* digunakan untuk mengatur perubahan nilai *gap*, sedangkan perulangan *for* digunakan untuk memproses setiap elemen *array*. Proses pergeseran elemen dilakukan menggunakan perulangan *while* sehingga data dapat tersusun dalam urutan yang benar.

3. Membuat *method main*

```

public static void main(String[] args) {
    int [ ] data_2002= {3, 10, 4, 6, 8, 9, 7, 2, 1, 5};
    System.out.print("Sebelum: ");
    printArray_2002(data_2002);

    shellSort_2511532002(data_2002);

    System.out.print("Sesudah (ShellSort): ");
    printArray_2002(data_2002);
}

```

Method main digunakan sebagai *method* utama untuk *Method main()* digunakan sebagai *method* utama untuk menjalankan program. Pada *method* ini dibuat sebuah *array* yang berisi data yang belum terurut. Selanjutnya program menampilkan isi *array* sebelum proses pengurutan, kemudian memanggil *method shellSort_2511532002()* untuk melakukan pengurutan data. Setelah proses selesai, program akan menampilkan *array* yang telah terurut menggunakan algoritma *Shell Sort*.

4. Membuat *Method* Print Array

```
public static void printArray_2002(int[] arr) {
    for (int i_2002:arr) System.out.print(i_2002 + " ");
    System.out.println();
}
```

Digunakan untuk menampilkan isi *array* ke layar. *Method* ini menggunakan perulangan *for* untuk menampilkan setiap elemen array secara berurutan

2.2.3 Membuat Program *QuickSort_2511532002*

```
package pekan8_2511532002;

public class QuickSort_2511532002 {
    static void swap_2002 (int[] arr_2002, int i_2002, int j_2002) {
        int temp_2002=arr_2002[i_2002];
        arr_2002[i_2002]=arr_2002[j_2002];
        arr_2002[j_2002]=temp_2002;
    }
    //Metode tambahan untuk mengatur pivot menggunakan Median-of-Three
    static void medianOfThree_2002 (int[] arr_2002, int low_2002, int high_2002) {
        int mid_2002=low_2002+(high_2002-low_2002)/2;

        //urutan elemen low, mid, dan high
        if (arr_2002[low_2002]>arr_2002[mid_2002]) {
            swap_2002(arr_2002, low_2002, mid_2002);
        }
        if (arr_2002[low_2002]> arr_2002[high_2002]) {
            swap_2002(arr_2002, low_2002, high_2002);
        }
        if (arr_2002[mid_2002]> arr_2002[high_2002]) {
            swap_2002(arr_2002, mid_2002, high_2002);
            swap_2002 (arr_2002, mid_2002, high_2002);
        }
    }
    static int partition(int[] arr_2002, int low_2002, int high_2002)
    {
        //panggil fungsi medianOfThree sebelum menentukan pivot
        medianOfThree_2002(arr_2002, low_2002, high_2002);

        int pivot_2002=arr_2002[high_2002];
        int i_2002 = (low_2002 - 1);

        for (int j_2002 =low_2002; j_2002 <= high_2002 - 1; j_2002++) {
            //jika elemen saat ini lebih kecil dari
            //atau sama dengan pivot
            //lebih kecil
            if (arr_2002[j_2002]<pivot_2002) {
                //increment indeks elemen yang
                i_2002++;
                swap_2002(arr_2002, i_2002, j_2002);
            }
        }
        swap_2002(arr_2002, i_2002+1, high_2002);
        return (i_2002+1);
    }
}
```



```

static void quickSort_2002(int[] arr_2002, int low_2002, int
high_2002) {
    if (low_2002<high_2002) {
        int pi_2002=partition(arr_2002, low_2002, high_2002);

        quickSort_2002(arr_2002, low_2002, pi_2002-1);
        quickSort_2002(arr_2002, pi_2002+1, high_2002);
    }

    public static void printArr_2002(int[] arr_2002) {
        for (int i_2002=0; i_2002<arr_2002.length; i_2002++) {
            System.out.print(arr_2002[i_2002]+" ");
        }
        System.out.println();
    }

    public static void main(String[] args) {
        int[] arr_2002 = {10, 7, 8, 9, 1, 5};
        int n_2002 =arr_2002.length;
        System.out.print("Data sebelum diurutkan: ");
        printArr_2002(arr_2002);

        quickSort_2002(arr_2002, 0, n_2002 - 1);

        System.out.print("Data Terurut QuickSort: ");
        printArr_2002(arr_2002);
    }
}

```

1. Membuat *class Quick Sort*

```
public class QuickSort_2511532002 {
```

Syntax di atas digunakan untuk membuat *class* bernama *QuickSort_2511532002* sebagai tempat penulisan program algoritma *Quick Sort*.

2. Membuat *method Swap*

```

static void swap_2002 (int[] arr_2002, int i_2002, int j_2002) {
    int temp_2002=arr_2002[i_2002];
    arr_2002[i_2002]=arr_2002[j_2002];
    arr_2002[j_2002]=temp_2002;
}

```

Digunakan untuk menukar posisi dua elemen dalam *array*. Menyimpan sementara nilai pada *indeks* pertama ke *temp_2002*, kemudian menukar nilai kedua elemen tersebut. Method ini digunakan selama proses *partition* dan pemilihan *pivot*.

3. Membuat *method Median of Three*

```

static void medianOfThree_2002 (int[] arr_2002, int low_2002, int
high_2002) {
    int mid_2002=low_2002+(high_2002-low_2002)/2;
}

```

```

//urutan elemen low, mid, dan high
if (arr_2002[low_2002]>arr_2002[mid_2002]) {
    swap_2002(arr_2002, low_2002, mid_2002);
}
if (arr_2002[low_2002]> arr_2002[high_2002]) {
    swap_2002(arr_2002, low_2002, high_2002);
}
if (arr_2002[mid_2002]> arr_2002[high_2002]) {
    swap_2002(arr_2002, mid_2002, high_2002);
    swap_2002 (arr_2002, mid_2002, high_2002);
}
}

```

Method medianOfThree_2002() digunakan untuk menentukan pivot menggunakan teknik *Median of Three*. Teknik ini memilih nilai tengah dari tiga elemen, yaitu elemen pertama (*low*), elemen tengah (*mid*), dan elemen terakhir (*high*). Tujuannya adalah memperoleh *pivot* yang lebih baik sehingga dapat meningkatkan performa *Quick Sort* dan mengurangi kemungkinan terjadinya kasus terburuk.

4. Membuat *Method Partition*

```

static int partition(int[] arr_2002, int low_2002, int high_2002) {
//panggil fungsi medianOfThree sebelum menentukan pivot
medianOfThree_2002(arr_2002, low_2002, high_2002);
int pivot_2002=arr_2002[high_2002];
int i_2002 = (low_2002 -1);
for (int j_2002 =low_2002; j_2002 <= high_2002 - 1; j_2002++) {
    //Jika elemen saat ini lebih kecil dari atau sama dengan pivot
    if (arr_2002[j_2002]<pivot_2002) {
        //increment indeks elemen yang lebih kecil
        i_2002++;
        swap_2002(arr_2002, i_2002, j_2002);
    }
}
swap_2002(arr_2002, i_2002+1, high_2002);
return (i_2002+1);
}

```

Method partition() digunakan untuk membagi *array* menjadi dua bagian berdasarkan nilai *pivot*. Sebelum proses pembagian dilakukan, *method medianOfThree_2002()* dipanggil terlebih dahulu untuk menentukan *pivot* yang digunakan.

Selanjutnya program akan membandingkan setiap elemen dengan *pivot*. Jika elemen lebih kecil dari *pivot*, maka elemen tersebut dipindahkan ke sisi kiri *array*. Setelah seluruh elemen diperiksa, *pivot* ditempatkan pada posisi yang tepat dan *method* mengembalikan indeks *pivot* tersebut.

5. Membuat *methode Quick Sort*

```
static void quickSort_2002(int[] arr_2002, int low_2002, int high_2002) {
    if (low_2002 < high_2002) {
        int pi_2002 = partition(arr_2002, low_2002, high_2002);

        quickSort_2002(arr_2002, low_2002, pi_2002 - 1);
        quickSort_2002(arr_2002, pi_2002 + 1, high_2002);
    }
}
```

Method quickSort_2002() digunakan untuk melakukan proses pengurutan data menggunakan algoritma *Quick Sort*. *Method* ini bekerja secara rekursif dengan membagi array menjadi dua bagian menggunakan *method partition()*, kemudian mengurutkan bagian kiri dan kanan secara terpisah hingga seluruh data terurut.

Proses rekursif akan berhenti ketika jumlah elemen yang diproses hanya tersisa satu atau tidak ada lagi elemen yang perlu diurutkan.

6. Membuat *methode Print Array*

```
public static void printArr_2002(int[] arr_2002) {
    for (int i_2002 = 0; i_2002 < arr_2002.length; i_2002++) {
        System.out.print(arr_2002[i_2002] + " ");
    }
    System.out.println();
}
```

PrintArr_2002() digunakan untuk menampilkan isi *array* ke layar. *Method* ini menggunakan perulangan *for* untuk menampilkan setiap elemen *array*.

7. Membuat *methode Main*

```
public static void main(String[] args) {
    int[] arr_2002 = {10, 7, 8, 9, 1, 5};
    int n_2002 = arr_2002.length;
    System.out.print("Data sebelum diurutkan: ");
    printArr_2002(arr_2002);

    quickSort_2002(arr_2002, 0, n_2002 - 1);

    System.out.print("Data Terurut QuickSort: ");
    printArr_2002(arr_2002);
}
```

Method main() digunakan sebagai *method* utama untuk menjalankan program. Pada *method* ini dibuat sebuah *array* yang berisi data yang belum terurut. Program kemudian menampilkan data sebelum

proses pengurutan, memanggil *method quickSort_2002()* untuk mengurutkan data menggunakan algoritma *Quick Sort*, dan menampilkan kembali data yang telah terurut setelah proses pengurutan selesai dilakukan.

2.2.4 Membuat Program *MergeSort_2511532002* {

```
package pekan8_2511532002;

public class MargeSort_2511532002 {
    void merge_2002(int arr_2002[], int l_2002, int m_2002, int
r_2002) {
        //Find sizes of two subarrays to be merged
        int n1_2002=m_2002-l_2002+1;
        int n2_2002=r_2002-m_2002;

        /*Create temp arrays*/
        int L_2002[] = new int[n1_2002];
        int R_2002[] = new int[n2_2002];

        /* Copy data to temp arrays */
        for (int i_2002=0; i_2002 < n1_2002; ++i_2002)
            L_2002[i_2002]=arr_2002[l_2002+i_2002];
        for (int j_2002=0; j_2002 < n2_2002; ++j_2002)
            R_2002[j_2002]=arr_2002[m_2002+1+j_2002];
        int i_2002=0, j_2002=0;

        //Initial inde of merged subarray array
        int k_2002=l_2002;
        while (i_2002 < n1_2002 && j_2002 < n2_2002) {
            if (L_2002[i_2002] <= R_2002[j_2002]) {
                arr_2002[k_2002] = L_2002[i_2002];
                i_2002++;
            } else {
                arr_2002[k_2002] = R_2002[j_2002];
                j_2002++;
            }
            k_2002++;
        }
        /*Copy remaining elements of L_2002[] if any */
        while (i_2002 < n1_2002) {
            arr_2002[k_2002] = L_2002[i_2002];
            i_2002++;
            k_2002++;
        }

        /*Copy remaining elements of R_2002[] if any*/
        while (j_2002 < n2_2002) {
            arr_2002[k_2002] = R_2002[j_2002];
            j_2002++;
            k_2002++;
        }
    }

    void sort_2002 (int arr_2002[], int l_2002, int r_2002) {
        if (l_2002 < r_2002) {
            //Find the middle point
            int m_2002 = (l_2002 + r_2002)/2;
            //Sort first
            sort_2002(arr_2002, l_2002, m_2002);
            sort_2002(arr_2002, m_2002 + 1, r_2002);

            // Merge the sorted halves
            merge_2002(arr_2002, l_2002, m_2002, r_2002);
        }
    }

    /* A utility function to print array of size n */
}
```

```

static void printArray(int arr_2002[]) {
    int n_2002 = arr_2002.length;

    for (int i_2002 = 0; i_2002 < n_2002; ++i_2002)
        System.out.print(arr_2002[i_2002] + " ");

    System.out.println();
}

public static void main(String args_2002[]) {

    int arr_2002[] = {12, 11, 13, 5, 6, 7};

    System.out.println("Sebelum terurut: ");
    printArray(arr_2002);

    MargeSort_2511532002 ob_2002 = new
MargeSort_2511532002();

    ob_2002.sort_2002(arr_2002, 0, arr_2002.length - 1);

    System.out.println("\nSesudah Terurut menggunakan Merge
Sort: ");
    printArray(arr_2002);
}
}

```

1. Membuat class Merge Sort

```
public class BubleSort_2511532002 {
```

Syntax di atas digunakan untuk membuat class bernama *MergeSort_2511532002* sebagai tempat penulisan program algoritma Merge Sort.

2. Membuat method Bubble Sort

```
public static void BubleSort_2511532002(int[] arr) {
    void merge_2002(int arr_2002[], int l_2002, int m_2002, int r_2002) {
```

//Find sizes of two subarrays to be merged

```
int n1_2002=m_2002-l_2002+1;
```

```
int n2_2002=r_2002-m_2002;
```

*/*Create temp arrays*/*

```
int L_2002[] = new int[n1_2002];
```

```
int R_2002[] = new int [n2_2002];
```

/ Copy data to temp arrays */*

```
for (int i_2002=0; i_2002 < n1_2002; ++i_2002)
```

```
    L_2002[i_2002]=arr_2002[l_2002+i_2002];
```

```
for (int j_2002 =0; j_2002 <n2_2002; ++j_2002)
```

```
    R_2002[j_2002]=arr_2002[m_2002+1+j_2002];
```

```
int i_2002=0, j_2002=0;
```

//Initial inde of merged subarray array

```

int k_2002=l_2002;
while (i_2002 <n1_2002&& j_2002<n2_2002) {
    if (L_2002[i_2002] <= R_2002[j_2002]) {
        arr_2002 [k_2002] =L_2002[i_2002];
        i_2002++;
    }else {
        arr_2002[k_2002]=R_2002[j_2002];
        j_2002++;
    }
    k_2002++;
}
/*Copy remaining elements of L_2002[] if any */
while (i_2002 <n1_2002) {
    arr_2002[k_2002]= L_2002[i_2002];
    i_2002++;
    k_2002++;
}
/*Copy remaining elements of R_2002[] if any*/
while (j_2002 < n2_2002) {
    arr_2002[k_2002]=R_2002[j_2002];
    j_2002++;
    k_2002++;
}

```

Membuat *Method merge_2002()* digunakan untuk menggabungkan dua *subarray* yang telah terurut menjadi satu *array* yang tetap terurut. *Method* ini terlebih dahulu menentukan ukuran masing-masing *subarray*, kemudian membuat *array* sementara untuk menampung data dari bagian kiri dan kanan.

Selanjutnya, program membandingkan elemen dari kedua *subarray* satu per satu. Elemen yang memiliki nilai lebih kecil akan dimasukkan terlebih dahulu ke dalam *array* utama. Setelah salah satu *subarray* habis diproses, sisa elemen dari *subarray* lainnya akan langsung disalin ke *array* utama hingga seluruh data berhasil digabungkan.

3. Membuat *method Sort*

```
void sort_2002 (int arr_2002[], int l_2002, int r_2002) {
    if (l_2002 < r_2002) {
        //Find the middle point
        int m_2002 = (l_2002 + r_2002)/2;
        //Sort first
        sort_2002(arr_2002, l_2002, m_2002);
        sort_2002(arr_2002, m_2002 + 1, r_2002);

        // Merge the sorted halves
        merge_2002(arr_2002, l_2002, m_2002, r_2002);
    }
}
```

Digunakan untuk melakukan proses pengurutan data menggunakan algoritma *Merge Sort*. Method ini bekerja secara rekursif dengan membagi *array* menjadi dua bagian yang lebih kecil hingga setiap bagian hanya memiliki satu elemen.

Setelah proses pembagian selesai, *method merge_2002()* dipanggil untuk menggabungkan kembali subarray-subarray tersebut secara berurutan sehingga menghasilkan *array* yang telah terurut.

4. Membuat *methode Print Array*

```
static void printArray(int arr_2002[]) {
    int n_2002 = arr_2002.length;

    for (int i_2002 = 0; i_2002 < n_2002; ++i_2002)
        System.out.print(arr_2002[i_2002] + " ");

    System.out.println();
}
```

Digunakan untuk menampilkan isi *array* ke layar. *Method* ini menggunakan perulangan *for* untuk menampilkan setiap elemen *array* secara berurutan.

5. Membuat *methode Main*

```
public static void main(String args_2002[]) {
    int arr_2002[] = {12, 11, 13, 5, 6, 7};
    System.out.println("Sebelum terurut: ");
    printArray(arr_2002);
    MargeSort_2511532002 ob_2002 = new MargeSort_2511532002();
    ob_2002.sort_2002(arr_2002, 0, arr_2002.length - 1);
    System.out.println("\nSesudah Terurut menggunakan Merge Sort: ");
    printArray(arr_2002);
}
```

Method main() digunakan sebagai *method* utama untuk menjalankan program. Pada *method* ini dibuat sebuah *array* yang berisi data yang belum terurut, kemudian menampilkan isi array sebelum proses pengurutan, membuat objek dari class *MargeSort_2511532002*, dan memanggil *method sort_2002()* untuk melakukan pengurutan dan kemudain menampilkan kembali isi *array* setelah pengurutan.

2.2.5 Membuat Program *BubleSortGUI_2511532002*

```
package pekan8_2511532002;

import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.EventQueue;
import java.awt.FlowLayout;
import java.awt.Font;

import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;
import javax.swing.JTextField;
import javax.swing.SwingConstants;
import javax.swing.SwingUtilities;
import javax.swing.border.EmptyBorder;

import pekan8_2511532002.BubleSortGUI_2511532002;

public class BubleSortGUI_2511532002 extends JFrame {

    private static final long serialVersionUID = 1L;
    private int[] array_2002;
    private JLabel[] labelArray_2002;
    private JButton stepButton_2002, resetButton_2002, setButton_2002;
    private JTextField inputField_2002;
    private JPanel panelArray_2002;
    private JTextArea stepArea_2002;

    private int i= 0;
    private int j=0;
    private boolean sorting = false;
    private int stepCount=1;
    private JPanel contentPane;

    /**
     * Launch the application.
     */

    public BubleSortGUI_2511532002() {
        setTitle("BubleSort Langkah per Langkah");
        setSize(750, 400);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout());

        // Panel Input
        JPanel inputPanel = new JPanel(new FlowLayout());
        inputField_2002 = new JTextField(30);
        setButton_2002 = new JButton("Set Array");

        inputPanel.add(new JLabel("Masukan angka (pisahkan dengan koma):"));
    });

    inputPanel.add(inputField_2002);
    inputPanel.add(setButton_2002);

    // Panel array visual
    panelArray_2002 = new JPanel();
    panelArray_2002.setLayout(new FlowLayout());

    // Panel Kontrol
    JPanel controlPanel = new JPanel();
    stepButton_2002 = new JButton("Langkah Selanjutnya");
    resetButton_2002 = new JButton("Reset");
```

```

        stepButton_2002.setEnabled(false);

        controlPanel.add(stepButton_2002);
        controlPanel.add(resetButton_2002);

        // Area teks untuk log langkah-langkah
        stepArea_2002 = new JTextArea(8, 60);
        stepArea_2002.setEditable(false);
        stepArea_2002.setFont(new Font("Monospaced", Font.PLAIN, 14));

        JScrollPane scrollPane = new JScrollPane(stepArea_2002);

        //Tambahan Panel ke frame
        add(inputPanel, BorderLayout.NORTH);
        add(panelArray_2002, BorderLayout.CENTER);
        add(controlPanel, BorderLayout.SOUTH);
        add(scrollPane, BorderLayout.EAST);

        //Event set array
        setButton_2002.addActionListener(e-> setArrayFromInput());

        //Event langkah selanjutnya
        stepButton_2002.addActionListener(e-> performStep());

        //Event reset
        resetButton_2002.addActionListener(e-> reset ());
    }

    private void setArrayFromInput() {
        String text = inputField_2002.getText().trim();
        if (text.isEmpty()) return;
        String [] parts=text.split(",");
        array_2002=new int[parts.length];
        try {
            for (int k=0; k< parts.length; k++) {
                array_2002[k]
=Integer.parseInt(parts[k].trim());}
            }catch (NumberFormatException e) {
                JOptionPane.showMessageDialog(this, "Masukan hanya angka
yang dipisahkan"+"dengan koma!", "Error", JOptionPane.ERROR_MESSAGE);
                return;
            }
            i=0;
            j=0;
            stepCount=1;
            sorting=true;
            stepButton_2002.setEnabled(true);
            stepArea_2002.setText("");
            panelArray_2002.removeAll();
            labelArray_2002=new JLabel[array_2002.length];
            for (int k=0; k < array_2002.length; k++) {
                labelArray_2002[k] = new
JLabel(String.valueOf(array_2002[k]));
                labelArray_2002[k].setFont(new Font ("Arial", Font.BOLD,
24));

                labelArray_2002[k].setOpaque(true);
                labelArray_2002[k].setBackground(Color.white);

            labelArray_2002[k].setBorder(BorderFactory.createLineBorder(Color.BLACK))
;
                labelArray_2002[k].setPreferredSize(new Dimension(50,
50));

            labelArray_2002[k].setHorizontalAlignment(SwingConstants.CENTER);
            panelArray_2002.add(labelArray_2002[k]);
        }
        panelArray_2002.revalidate();
        panelArray_2002.repaint();
    }

    private void performStep() {

```

```

        if (i < array_2002.length && sorting) {
            int key = array_2002[i];
            j = i - 1;

            StringBuilder stepLog = new StringBuilder();
            stepLog.append("Langkah").append(stepCount).append(":
Memasukan ").append(key).append("\n");

            while (j >= 0 && array_2002[j] > key) {
                array_2002[j + 1] = array_2002[j];
                j--;
            }

            array_2002[j + 1] = key;

            updateLabels();
            stepLog.append("Hasil:
").append(arrayToString(array_2002)).append("\n\n");
            stepArea_2002.append(stepLog.toString());

            i++;
            stepCount++;

            if (i == array_2002.length) {
                sorting = false;
                stepButton_2002.setEnabled(false);
                JOptionPane.showMessageDialog(this, "Sorting
selesai!");
            }
        }
    }

    private void updateLabels() {
        for (int k = 0; k < array_2002.length; k++) {
            labelArray_2002[k].setText(String.valueOf(array_2002[k]));
        }
    }

    private void resetHighlights() {
        for (JLabel label : labelArray_2002) {
            label.setBackground(Color.white);
        }
    }

    private void reset() {
        inputField_2002.setText("");
        panelArray_2002.removeAll();
        panelArray_2002.revalidate();
        panelArray_2002.repaint();
        stepArea_2002.setText("");
        stepButton_2002.setEnabled(false);
        sorting = false;
        i = 0;
        j = 0;
        stepCount = 1;
    }

    private String arrayToString(int[] arr) {
        StringBuilder sb = new StringBuilder();

        for (int k = 0; k < arr.length; k++) {
            sb.append(arr[k]);

            if (k < arr.length - 1)
                sb.append(", ");
        }

        return sb.toString();
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            BubleSortGUI_2511532002 gui =

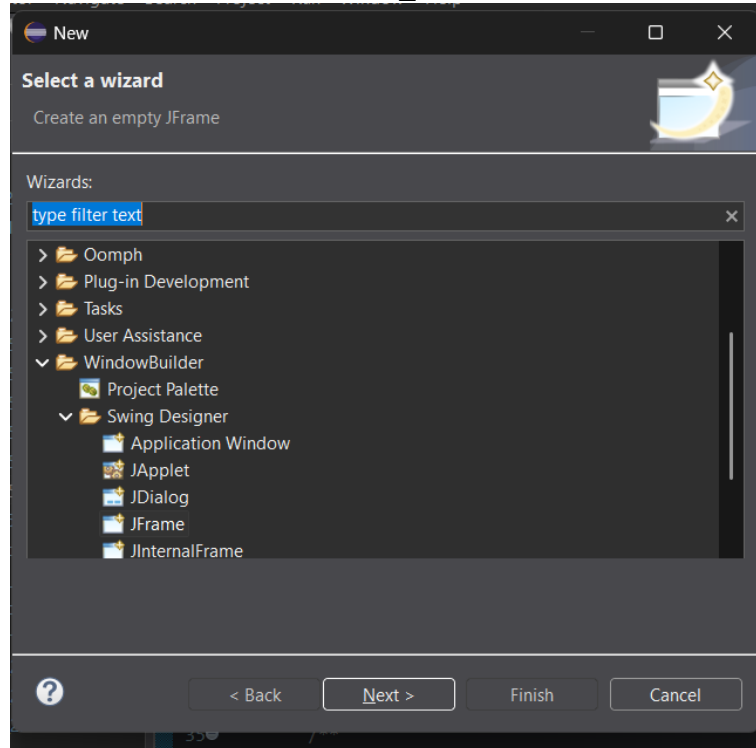
```

```

        new BubleSortGUI_2511532002();
        gui.setVisible(true);
    });
}
}

```

1. Membuat kelas *BubleSortGUI_2511532002*



Klik kanan pada *package*, lalu pilih “New” lalu pilih “Other”. Selanjutnya pilih “JFrame”.

2. Menulis syntax program, untuk menulis *syntax* buka tab “Source” maka akan terbuka tab *syntax* program *InsertionSortGUI_2511532002*.

3. Membuat class *GUI Insertion Sort* `public class BubleSortGUI_2511532002 extends JFrame {` Syntax di atas digunakan untuk membuat class GUI bernama *BubleSortGUI_2511532002* yang mewarisi *JFrame* sebagai tampilan utama program visualisasi *Insertion Sort*.

4. Membuat *atribut* dan komponen *GUI*

```

private int[] array_2002;
private JLabel[] labelArray_2002;
private JButton stepButton_2002, resetButton_2002, setButton_2002;
private JTextField inputField_2002;
private JPanel panelArray_2002;
private JTextArea stepArea_2002;

```

Syntax di atas digunakan untuk membuat atribut program dan komponen *GUI* seperti tombol, panel, *text field*, label, dan area teks yang digunakan untuk menampilkan visualisasi proses *sorting*.

5. Membuat *constructor GUI*

```
public BubleSortGUI_2511532002() {
```

Constructor digunakan untuk mengatur tampilan *GUI* seperti ukuran *frame*, *layout*, panel *input*, panel visualisasi array, tombol kontrol, dan area log langkah-langkah *sorting*.

6. Membuat *methode Set Array*

```
private void setArrayFromInput() {
    String text = inputField_2002.getText().trim();
    if (text.isEmpty()) return;
    String [] parts=text.split(",");
    array_2002=new int[parts.length];
    try {
        for (int k=0; k< parts.length; k++) {
            array_2002[k]
=Integer.parseInt(parts[k].trim());
        }catch (NumberFormatException e) {
            JOptionPane.showMessageDialog(this, "Masukan
hanya angka yang dipisahkan"+"dengan koma!", "Error",
JOptionPane.ERROR_MESSAGE);
            return;
        }
        i=0;
        j=0;
        stepCount=1;
        sorting=true;
        stepButton_2002.setEnabled(true);
        stepArea_2002.setText("");
        panelArray_2002.removeAll();
        labelArray_2002=new JLabel[array_2002.length];
        for (int k=0; k < array_2002.length; k++) {
            labelArray_2002[k] = new
JLabel(String.valueOf(array_2002[k]));
            labelArray_2002[k].setFont(new Font ("Arial",
Font.BOLD, 24));
            labelArray_2002[k].setOpaque(true);
            labelArray_2002[k].setBackground(Color.white);

            labelArray_2002[k].setBorder(BorderFactory.createLineBorder(Color
.BLACK));
            labelArray_2002[k].setPreferredSize(new
Dimension(50, 50));

            labelArray_2002[k].setHorizontalAlignment(SwingConstants.CENTER);
            panelArray_2002.add(labelArray_2002[k]);
        }
        panelArray_2002.revalidate();
        panelArray_2002.repaint();
    }
}
```

Method *setArrayFromInput()* digunakan untuk membaca data yang dimasukkan pengguna melalui *textbox*, kemudian mengubahnya menjadi *array* bertipe *integer*. Setelah itu, data ditampilkan dalam bentuk label pada panel visualisasi.

7. Membuat *methode Perform Step*

```
private void performStep() {
    if (i < array_2002.length && sorting) {
        int key = array_2002[i];
        j = i - 1;

        StringBuilder stepLog = new StringBuilder();

        stepLog.append("Langkah").append(stepCount).append(": Memasukan ")
            .append(key).append("\n");

        while (j >= 0 && array_2002[j] > key) {
            array_2002[j + 1] = array_2002[j];
            j--;
        }
        array_2002[j + 1] = key;

        updateLabels();

        stepLog.append("Hasil:").append(arrayToString(array_2002)).append(
            "\n\n");
        stepArea_2002.append(stepLog.toString());

        i++;
        stepCount++;
        if (i == array_2002.length) {
            sorting = false;
            stepButton_2002.setEnabled(false);
            JOptionPane.showMessageDialog(this, "Sorting
selesai!");
        }
    }
}
```

Method *performStep()* digunakan untuk menjalankan proses sorting secara bertahap setiap kali tombol Langkah Selanjutnya ditekan. Method ini menggunakan konsep *Insertion Sort*, yaitu mengambil satu elemen sebagai *key*, kemudian menyisipkannya pada posisi yang sesuai hingga data menjadi terurut.

8. Membuat *methode Update Labels*

```
private void updateLabels() {
    for (int k = 0; k < array_2002.length; k++) {

        labelArray_2002[k].setText(String.valueOf(array_2002[k]));
    }
}
```

Method updateLabels() digunakan untuk memperbarui tampilan nilai pada setiap label setelah terjadi perubahan data selama proses sorting sehingga pengguna dapat melihat hasil pengurutan secara langsung.

9. Membuat *methode Reset*

```
private void reset() {
    inputField_2002.setText("");
    panelArray_2002.removeAll();
    panelArray_2002.revalidate();
    panelArray_2002.repaint();
    stepArea_2002.setText("");
    stepButton_2002.setEnabled(false);
    sorting = false;
    i = 0;
    j=0;
    stepCount = 1;
}
```

Method reset() digunakan untuk mengembalikan program ke kondisi awal dengan menghapus data *input*, membersihkan tampilan visualisasi, serta mengatur ulang proses *sorting*.

10. Membuat *method main*

```
public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        BubleSortGUI_2511532002 gui =
            new BubleSortGUI_2511532002();
        gui.setVisible(true);
    });
}
```

Method main digunakan sebagai *method* utama untuk menjalankan program *GUI*. *Method* ini membuat objek *BubleSortGUI_2511532002* dan menampilkan jendela aplikasi sehingga pengguna dapat melakukan visualisasi proses *sorting*.

2.2.6 Membuat Program *MergeSortGUI_2511532002*

```
package pekan8_2511532002;

import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.EventQueue;
import java.awt.FlowLayout;
import java.awt.Font;

import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;
import javax.swing.JTextField;
import javax.swing.SwingConstants;
import javax.swing.SwingUtilities;
import javax.swing.border.EmptyBorder;
import java.util.LinkedList;
import java.util.Queue;

public class MergeSortGUI_2511532002 extends JFrame {
    private static final long serialVersionUID = 1L;
    private JPanel contentPane_2002;
    private int[] array_2002;
    private JLabel[] labelArray_2002;
    JButton stepButton_2002;
    JButton resetButton_2002;
    JButton setButton_2002;
    private JTextField inputField_2002;
    private JPanel panelArray_2002;
    private JTextArea stepArea_2002;
    private int i_2002 = 1, j_2002;
    private boolean sorting_2002 = false;
    private int stepCount_2002 = 1;
    private Queue<int[]> mergeQueue_2002 = new LinkedList<>();
    private boolean isMerging_2002 = false;
    private boolean copying_2002 = false;
    private int left_2002;
    private int mid_2002;
    private int right_2002;
    private int k_2002;
    private int[] temp_2002;

    /**
     * Create the frame.
     * @return
     */

    public MergeSortGUI_2511532002() {
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setBounds(100, 100, 450, 300);
        contentPane_2002 = new JPanel();
        contentPane_2002.setBorder(new EmptyBorder(5, 5, 5, 5));
        setContentPane(contentPane_2002);
        contentPane_2002.setLayout(null);
        setTitle("Merge sort langkah per langkah");
        setSize(750, 400);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout());

        // Panel input
        JPanel inputPanel = new JPanel(new FlowLayout());
        inputField_2002 = new JTextField(30);
        setButton_2002 = new JButton("Set Array");
```



```

        inputPanel.add(new JLabel("Masukkan angka (pisahkan
dengan koma): "));
        inputPanel.add(inputField_2002);
        inputPanel.add(setButton_2002);

        // Panel Array
        panelArray_2002 = new JPanel();
        panelArray_2002.setLayout(new FlowLayout());

        // Panel kontrol
        JPanel controlPanel = new JPanel();
        stepButton_2002 = new JButton("Langkah Selanjutnya");
        resetButton_2002 = new JButton("Reset");
        stepButton_2002.setEnabled(false);
        controlPanel.add(stepButton_2002);
        controlPanel.add(resetButton_2002);

        // Teks untuk log langkah-langkah
        stepArea_2002 = new JTextArea(8, 60);
        stepArea_2002.setEditable(false);
        stepArea_2002.setFont(new Font("Monospaced", Font.PLAIN,
14));

        JScrollPane scrollPane = new JScrollPane(stepArea_2002);

        // Menambahkan Panel ke Frame
        add(inputPanel, BorderLayout.NORTH);
        add(panelArray_2002, BorderLayout.CENTER);
        add(controlPanel, BorderLayout.SOUTH);
        add(scrollPane, BorderLayout.EAST);

        setButton_2002.addActionListener(e ->
setArrayFromInput_2002());

        stepButton_2002.addActionListener(e ->
performStep_2002());

        // Even Reset
        resetButton_2002.addActionListener(e -> reset_2002());
    }

    private void setArrayFromInput_2002() {
        String text_2002 = inputField_2002.getText().trim();
        if (text_2002.isEmpty()) return;

        String[] parts_2002 = text_2002.split(",");
        array_2002 = new int[parts_2002.length];

        try {
            for (int i_2002 = 0; i_2002 < parts_2002.length;
i_2002++) {
                array_2002[i_2002] =
Integer.parseInt(parts_2002[i_2002].trim());
            }
        } catch (NumberFormatException e_2002) {
            JOptionPane.showMessageDialog(this, "Masukkan
hanya angka!", "Error", JOptionPane.ERROR_MESSAGE);
            return;
        }

        labelArray_2002 = new JLabel[array_2002.length];
        panelArray_2002.removeAll();

        for (int i_2002 = 0; i_2002 < array_2002.length;
i_2002++) {
            labelArray_2002[i_2002] = new
JLabel(String.valueOf(array_2002[i_2002]));
            labelArray_2002[i_2002].setFont(new Font("Arial",
Font.BOLD, 24));
            labelArray_2002[i_2002].setOpaque(true);

            labelArray_2002[i_2002].setBackground(Color.WHITE);

```

```

        labelArray_2002[i_2002].setBorder(BorderFactory.createLineBorder(
Color.BLACK));
        labelArray_2002[i_2002].setPreferredSize(new
Dimension(50, 50));

        labelArray_2002[i_2002].setHorizontalAlignment(SwingConstants.CEN
TER);

        panelArray_2002.add(labelArray_2002[i_2002]);

        mergeQueue_2002.clear();
        generateMergeSteps_2002(0, array_2002.length -
1);

        stepButton_2002.setEnabled(true);
        stepArea_2002.setText("");
        stepCount_2002 = 1;
        isMerging_2002 = false;
        panelArray_2002.revalidate();
        panelArray_2002.repaint();
    }
}

private void generateMergeSteps_2002(int left_2002, int
right_2002) {
    if (left_2002 >= right_2002) {
        return;
    }

    int mid_2002 = (left_2002 + right_2002) / 2;

    generateMergeSteps_2002(left_2002, mid_2002);
    generateMergeSteps_2002(mid_2002 + 1, right_2002);

    mergeQueue_2002.offer(new int[] {
        left_2002, mid_2002, right_2002
    });
}

private void performStep_2002() {
    resetHighlights_2002();

    if (!isMerging_2002 && !mergeQueue_2002.isEmpty()) {
        int[] range_2002 = mergeQueue_2002.poll();

        left_2002 = range_2002[0];
        mid_2002 = range_2002[1];
        right_2002 = range_2002[2];

        temp_2002 = new int[right_2002 - left_2002 + 1];

        i_2002 = left_2002;
        j_2002 = mid_2002 + 1;
        k_2002 = 0;

        copying_2002 = false;
        isMerging_2002 = true;

        stepArea_2002.append("Langkah " +
stepCount_2002++
        + ": Mulai merge dari " + left_2002 + "
ke "
        + right_2002 + "\n");
        return;
    }

    if (isMerging_2002 && !copying_2002) {
        if (i_2002 <= mid_2002 && j_2002 <= right_2002) {

```

```

labelArray_2002[i_2002].setBackground(Color.cyan);
labelArray_2002[j_2002].setBackground(Color.cyan);

        if (array_2002[i_2002] <= array_2002[j_2002])
        {
            temp_2002[k_2002++] =
array_2002[i_2002++];
        } else {
            temp_2002[k_2002++] =
array_2002[j_2002++];
        }

        stepArea_2002.append("Langkah " +
stepCount_2002++
            + ": Bandingkan dan salin elemen\n");
        return;

    } else if (i_2002 <= mid_2002) {
        temp_2002[k_2002++] = array_2002[i_2002++];
        stepArea_2002.append("Langkah " +
stepCount_2002++
            + ": Salin sisa kiri\n");
        return;

    } else if (j_2002 <= right_2002) {
        temp_2002[k_2002++] = array_2002[j_2002++];
        stepArea_2002.append("Langkah " +
stepCount_2002++
            + ": Salin sisa kanan\n");
        return;

    } else {
        copying_2002 = true;
        k_2002 = 0;
        return;
    }
}

if (copying_2002 && k_2002 < temp_2002.length) {
    array_2002[left_2002 + k_2002] =
temp_2002[k_2002];

    labelArray_2002[left_2002 + k_2002]
.setText(String.valueOf(temp_2002[k_2002]));

    labelArray_2002[left_2002 + k_2002]
.setBackground(Color.GREEN);

    k_2002++;

    stepArea_2002.append("Langkah " +
stepCount_2002++
        + ": Tempelkan ke array utama\n");
    return;
}

if (copying_2002 && k_2002 == temp_2002.length) {
    isMerging_2002 = false;
    copying_2002 = false;
}

if (mergeQueue_2002.isEmpty() && !isMerging_2002) {

```

```

        stepArea_2002.append("Selesai.\n");
        stepButton_2002.setEnabled(false);
        JOptionPane.showMessageDialog(this, "Merge Sort
selesai!");
    }
}

private void resetHighlights_2002() {
    if (labelArray_2002 == null) return;
    for (JLabel label_2002 : labelArray_2002) {
        label_2002.setBackground(Color.WHITE);
    }
}

private void reset_2002() {
    inputField_2002.setText("");

    panelArray_2002.removeAll();
    panelArray_2002.revalidate();
    panelArray_2002.repaint();

    stepArea_2002.setText("");

    stepButton_2002.setEnabled(false);

    mergeQueue_2002.clear();

    isMerging_2002 = false;
    stepCount_2002 = 1;
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        MergeSortGUI_2511532002 gui =
            new MergeSortGUI_2511532002();
        gui.setVisible(true);
    });
}
}

```

1. Membuat Atribut dan Komponen GUI

```

private int[] array_2002;
private JLabel[] labelArray_2002;
private JButton stepButton_2002, resetButton_2002,
setButton_2002;
private JTextField inputField_2002;
private JPanel panelArray_2002;
private JTextArea stepArea_2002;
private Queue<int[]> mergeQueue_2002;

```

Syntax di atas digunakan untuk membuat atribut program dan komponen GUI seperti tombol, panel, *text field*, *label*, area teks, serta *queue* yang digunakan untuk menyimpan tahapan proses *Merge Sort* yang akan divisualisasikan.

2. Membuat *Constructor GUI*

```
public MergeSortGUI_2511532002()
```

Constructor digunakan untuk mengatur tampilan *GUI* seperti ukuran frame, layout, panel input, panel visualisasi *array*, tombol kontrol, *area log* langkah-langkah *Merge Sort*, serta menghubungkan tombol dengan *method* yang akan dijalankan ketika pengguna berinteraksi dengan program

3. Membuat *Method Set Array*

```
private void setArrayFromInput_2002() {
    String text_2002 = inputField_2002.getText().trim();
    if (text_2002.isEmpty()) return;

    String[] parts_2002 = text_2002.split(",");
    array_2002 = new int[parts_2002.length];
    try {
        for (int i_2002 = 0; i_2002 < parts_2002.length; i_2002++) {
            array_2002[i_2002] = Integer.parseInt(parts_2002[i_2002].trim());
        }
    } catch (NumberFormatException e_2002) {
        JOptionPane.showMessageDialog(this, "Masukkan
        hanya angka!", "Error", JOptionPane.ERROR_MESSAGE);
        return;
    }

    labelArray_2002 = new JLabel[array_2002.length];
    panelArray_2002.removeAll();

    for (int i_2002 = 0; i_2002 < array_2002.length; i_2002++) {
        labelArray_2002[i_2002] = new
        JLabel(String.valueOf(array_2002[i_2002]));
    }
}
```

```

        labelArray_2002[i_2002].setFont(new Font("Arial",
Font.BOLD, 24));
        labelArray_2002[i_2002].setOpaque(true);
        labelArray_2002[i_2002].setBackground(Color.WHITE);
        labelArray_2002[i_2002].setBorder(BorderFactory.createLineB
order(Color.BLACK));
        labelArray_2002[i_2002].setPreferredSize(new Dimension(50,
50));
        labelArray_2002[i_2002].setHorizontalAlignment(SwingConsta
nts.CENTER);
        panelArray_2002.add(labelArray_2002[i_2002]);
        mergeQueue_2002.clear();
        generateMergeSteps_2002(0, array_2002.length - 1);
        stepButton_2002.setEnabled(true);
        stepArea_2002.setText("");
        stepCount_2002 = 1;
        isMerging_2002 = false;
        panelArray_2002.revalidate();
        panelArray_2002.repaint();
    }
}

```

Method *setArrayFromInput_2002()* digunakan untuk membaca data yang dimasukkan pengguna melalui *textbox*, kemudian memisahkan setiap angka berdasarkan tanda koma dan mengubahnya menjadi array bertipe integer. Setelah itu, data ditampilkan dalam bentuk label pada panel visualisasi. Method ini juga menginisialisasi proses *Merge Sort* dengan membuat daftar langkah penggabungan (*mergeQueue_2002*) yang akan dijalankan secara bertahap.

4. Membuat *Method Generate Merge Steps*

```
private void generateMergeSteps_2002(int left_2002, int right_2002)
{
    if (left_2002 >= right_2002) {
        return;
    }

    int mid_2002 = (left_2002 + right_2002) / 2;
    generateMergeSteps_2002(left_2002, mid_2002);
    generateMergeSteps_2002(mid_2002 + 1, right_2002);
    mergeQueue_2002.offer(new int[] {
        left_2002, mid_2002, right_2002
    });
}
```

Method *generateMergeSteps_2002()* digunakan untuk membagi array menjadi beberapa sub-array secara rekursif sesuai konsep algoritma *Merge Sort*. Setiap rentang data yang akan digabungkan disimpan ke dalam *mergeQueue_2002* sehingga proses merge dapat divisualisasikan langkah demi langkah saat program dijalankan.

5. Membuat *Method Perform Step*

```
private void performStep_2002()
```

Method *performStep_2002()* digunakan untuk menjalankan proses Merge Sort secara bertahap setiap kali tombol Langkah **Selanjutnya** ditekan. Method ini melakukan proses perbandingan elemen dari dua sub-array, menyalin elemen ke array sementara (*temp_2002*), kemudian menggabungkannya kembali ke array utama. Setiap langkah yang dilakukan akan dicatat pada area *log* dan ditampilkan secara visual melalui perubahan warna label.

6. Membuat *Method Reset Highlights*

```
private void resetHighlights_2002() {
    if (labelArray_2002 == null) return;
    for (JLabel label_2002 : labelArray_2002) {
        label_2002.setBackground(Color.WHITE);
    }
}
```

Method *resetHighlights_2002()* digunakan untuk mengembalikan warna seluruh label array menjadi warna putih sebelum langkah *Merge Sort* berikutnya dijalankan. Hal ini bertujuan agar elemen yang sedang dibandingkan pada setiap langkah dapat terlihat dengan lebih jelas.

7. Membuat *Method Reset*

```
private void reset_2002() {
    inputField_2002.setText("");
    panelArray_2002.removeAll();
    panelArray_2002.revalidate();
    panelArray_2002.repaint();

    stepArea_2002.setText("");
    stepButton_2002.setEnabled(false);
    mergeQueue_2002.clear();

    isMerging_2002 = false;
    stepCount_2002 = 1;
}
```

Method *reset_2002()* digunakan untuk mengembalikan program ke kondisi awal. Method ini menghapus data input, membersihkan panel visualisasi *array*, menghapus log langkah-langkah *Merge Sort*, menonaktifkan tombol proses, serta

mengatur ulang variabel yang digunakan selama proses pengurutan.

8. Membuat *Method Main*

```
public static void main(String[] args) {
    SwingUtilities.invokeLater() -> {
        MergeSortGUI_2511532002 gui =
            new MergeSortGUI_2511532002();

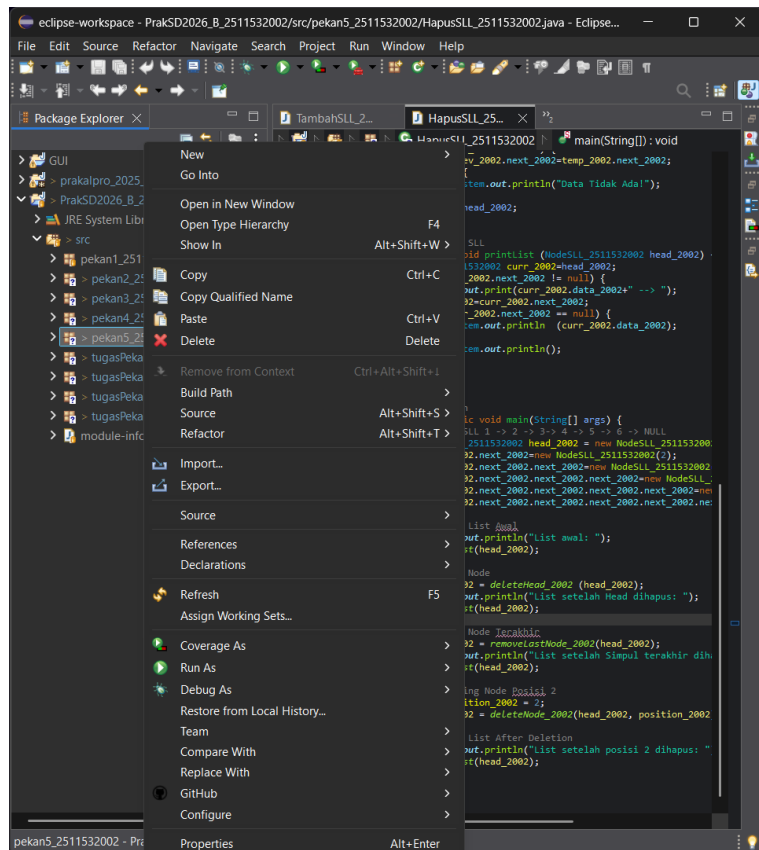
        gui.setVisible(true);
    });
}
```

Method main() digunakan sebagai method utama untuk menjalankan program *GUI*. Method ini membuat objek *MergeSortGUI_2511532002* dan menampilkan jendela aplikasi sehingga pengguna dapat melakukan visualisasi proses pengurutan data menggunakan algoritma *Merge Sort* secara langkah demi langkah.

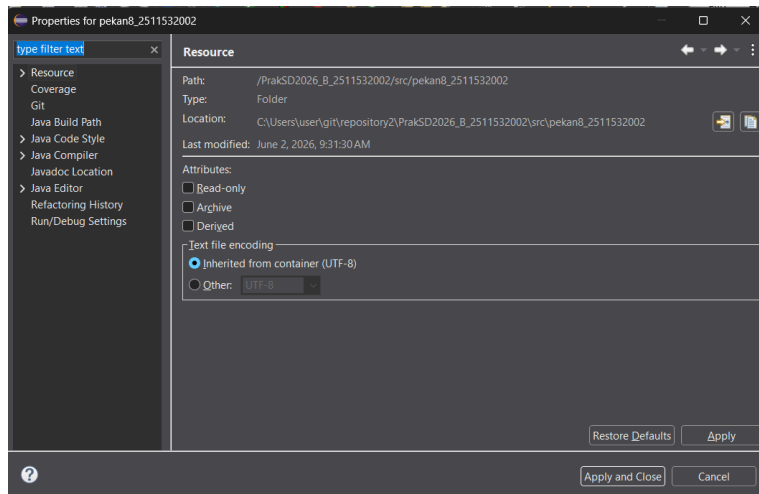
2.2.7 Upload ke GitHub

1. Setelah semua program selesai dibuat, selanjutnya adalah *user* perlu untuk memasukkan program yang dibuat di Eclipse ke *repository* GitHub. Selain dengan cara “*Team*” atau “*Commit and Push*”.

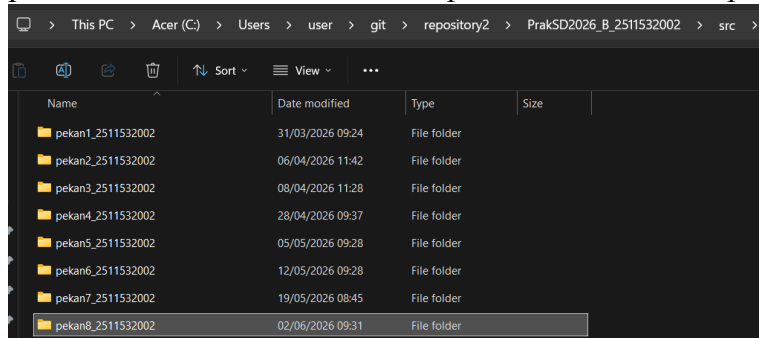
Bisa juga dengan cara *upload* manual. Caranya klik kanan pada *package*, lalu pilih *properties*.



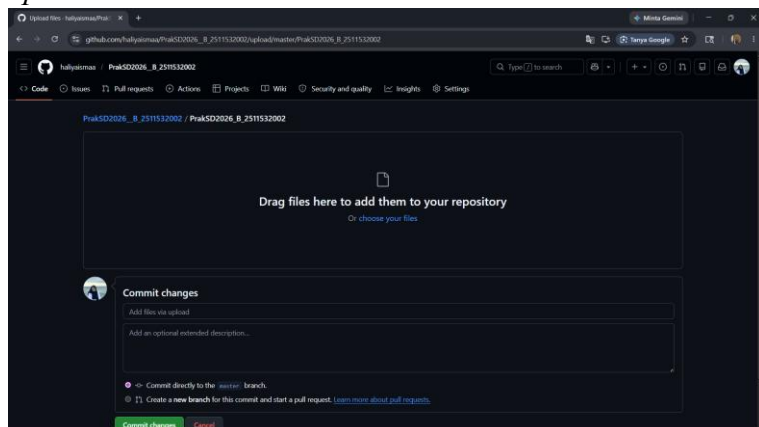
2. Maka akan muncul *pop up menu* “*Properteis for pekan8_2511532002*”. Lalu klik *icon* “*Show in System Folder*”.



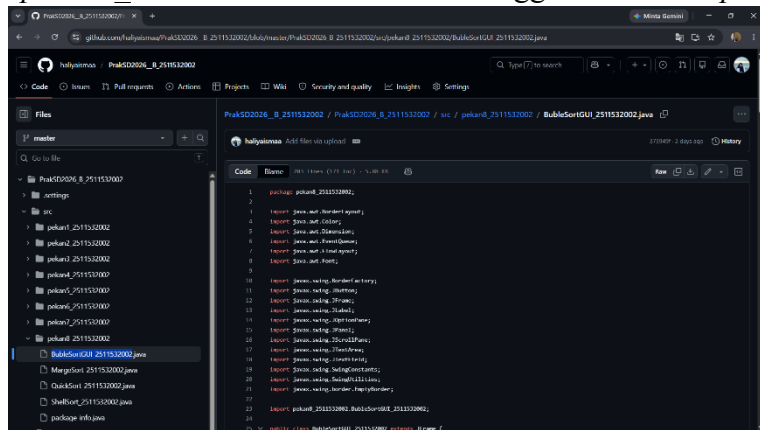
3. Maka akan menunjukan lokasi *file* “*pekan8_2511532002*” pada *File Explorer computer*.



4. Setelah itu *drag and drop file* “*pekan6_2511532002*” ke GitHub. Lalu tambahkan keterangan jika perlu. Setelah itu klik *upload*.



5. Maka dapat dilihat pada *repository GitHub* bahwa file “*pekan8_2511532002*” telah terunggah via *Upload*.



Dapat kita lihat disini jika kita buka, maka akan muncul *file package* dan *class* yang telah di buat di Eclipse.

2.3 Hasil

2.3.1 Hasil *ShellSort_2511532002*

```

1 package pekan8_2511532002;
2
3 public class ShellSort_2511532002 {
4     public static void shellSort_2511532002 (int[] A_2002) {
5         int n_2002= A_2002.length;
6         int gap_2002=n_2002/2;
7         while (gap_2002 > 0) {
8             for (int i_2002=gap_2002; i_2002<n_2002; i_2002++) {
9                 int temp_2002=A_2002[i_2002];
10                int j_2002=i_2002;
11                while (j_2002 >= gap_2002 && A_2002[j_2002-gap_2002]> temp_2002) {
12                    A_2002[j_2002]=A_2002[j_2002-gap_2002];
13                    j_2002=j_2002-gap_2002;
14                }
15                A_2002[j_2002]=temp_2002;
16            }
17            gap_2002=gap_2002/2;
18        }
19    }
20
21    public static void main(String[] args) {
22        int [] data_2002= {3, 10, 4, 6, 8, 9, 7, 2, 1, 5};
23        System.out.print("Sebelum: ");
24        printArray_2002(data_2002);
25
26        shellSort_2511532002(data_2002);
27
28        System.out.print("Sesudah (ShellSort): ");
29        printArray_2002(data_2002);
30    }
31
32    public static void printArray_2002(int[] arr) {
33        for (int i_2002=0; i_2002<arr.length; i_2002++) {
34            System.out.print(arr[i_2002] + " ");
35        }
36    }
37 }

```

Console Output:

```

<terminated> ShellSort_2511532002 [Java Application] C:\Users\user\p2\pool\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64
Sebelum: 3 10 4 6 8 9 7 2 1 5
Sesudah (ShellSort): 1 2 3 4 5 6 7 8 9 10

```

2.3.2 Hasil *QuickSort_2511532002*

```

36 //increment indeks elemen yang lebih kecil
37 i_2002++;
38 swap_2002(arr_2002, i_2002, j_2002);
39 }
40 swap_2002(arr_2002, i_2002+1, high_2002);
41 return (i_2002+1);
42 }
43
44 static void quickSort_2002(int[] arr_2002, int low_2002, int high_2002) {
45     if (low_2002<high_2002) {
46         int pi_2002=partition(arr_2002, low_2002, high_2002);
47         quickSort_2002(arr_2002, low_2002, pi_2002-1);
48         quickSort_2002(arr_2002, pi_2002+1, high_2002);
49     }
50 }
51
52 public static void printArr_2002(int[] arr_2002) {
53     for (int i_2002=0; i_2002<arr_2002.length; i_2002++) {
54         System.out.print(arr_2002[i_2002]+ " ");
55     }
56     System.out.println();
57 }
58
59 public static void main(String[] args) {
60     int[] arr_2002 = {10, 7, 8, 9, 1, 5};
61     int n_2002 = arr_2002.length;
62     System.out.print("Data sebelum diurutkan: ");
63     printArr_2002(arr_2002);
64
65     quickSort_2002(arr_2002, 0, n_2002 - 1);
66
67     System.out.print("Data Terurut QuickSort: ");
68     printArr_2002(arr_2002);
69 }
70 }
71

```

Console Output:

```

<terminated> QuickSort_2511532002 [Java Application] C:\Users\user\p2\pool\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64
Data sebelum diurutkan: 10 7 8 9 1 5
Data Terurut QuickSort: 1 5 7 8 9 10

```

2.3.3 Hasil MergeSort_2511532002

The screenshot shows the Eclipse IDE with the file `MergeSort_2511532002.java` open. The code implements a Merge Sort algorithm. The console output shows the initial array `12 11 13 5 6 7` and the sorted array `5 6 7 11 12 13`.

```

48      //Find the middle point
49      int m_2002 = (l_2002 + r_2002)/2;
50      //Sort first
51      sort_2002(arr_2002, l_2002, m_2002);
52      sort_2002(arr_2002, m_2002 + 1, r_2002);
53
54      // Merge the sorted halves
55      merge_2002(arr_2002, l_2002, m_2002, r_2002);
56  }
57
58
59  /* A utility function to print array of size n */
60  static void printArray(int arr_2002[]) {
61      int n_2002 = arr_2002.length;
62
63      for (int i_2002 = 0; i_2002 < n_2002; ++i_2002)
64          System.out.print(arr_2002[i_2002] + " ");
65
66      System.out.println();
67  }
68
69  public static void main(String args_2002[]) {
70
71      int arr_2002[] = {12, 11, 13, 5, 6, 7};
72
73      System.out.println("Sebelum terurut: ");
74      printArray(arr_2002);
75
76      MergeSort_2511532002 ob_2002 = new MergeSort_2511532002();
77
78      ob_2002.sort_2002(arr_2002, 0, arr_2002.length - 1);
79
80      System.out.println("\nSesudah Terurut menggunakan Merge Sort: ");
81      printArray(arr_2002);
82  }
83  }

```

Console Output:

```

<terminated> MergeSort_2511532002 [Java Application] C:\Users\user\p2\pool\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64
Sebelum terurut:
12 11 13 5 6 7

Sesudah Terurut menggunakan Merge Sort:
5 6 7 11 12 13

```

2.3.4 Hasil BubbleSortGUI_2511532002

The screenshot shows the Eclipse IDE with the file `BubbleSortGUI_2511532002.java` open. The code implements a Bubble Sort algorithm with a GUI. A dialog box titled "Bubble Sort GUI" is displayed, showing the array `1 2 3 4 5` and the message "Sorting selesai" (Sorting finished).

```

100  private void reset() {
101      for (int i = 0; i < arr.length; i++) {
102          arr[i] = i + 1;
103      }
104  }
105
106  private void sort() {
107      for (int i = 0; i < arr.length - 1; i++) {
108          for (int j = i + 1; j < arr.length; j++) {
109              if (arr[i] > arr[j]) {
110                  int temp = arr[i];
111                  arr[i] = arr[j];
112                  arr[j] = temp;
113              }
114          }
115      }
116  }
117
118  private void actionPerformed(ActionEvent e) {
119      if (e.getSource() == btnReset) {
120          reset();
121      } else if (e.getSource() == btnSort) {
122          sort();
123      }
124  }
125
126  public static void main(String[] args) {
127      SwingUtilities.invokeLater(() -> {
128          new BubbleSortGUI_2511532002().setVisible(true);
129      });
130  }

```

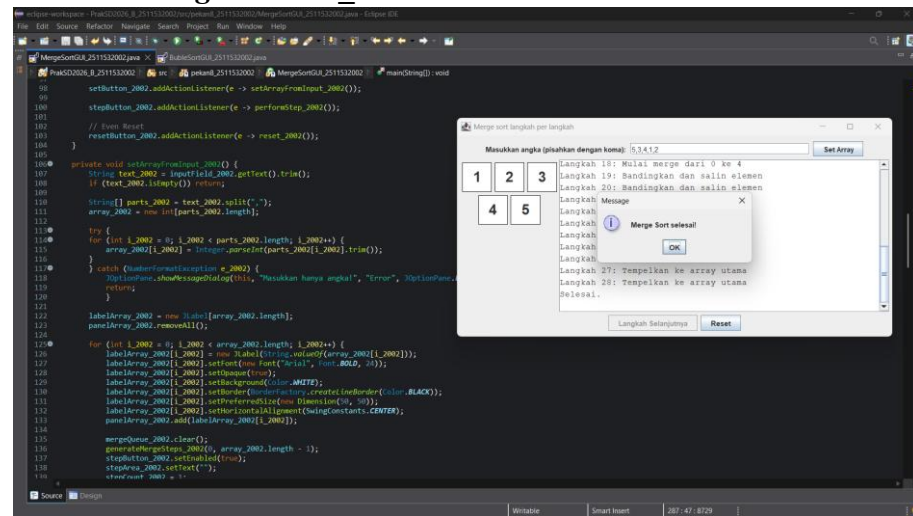
GUI Window Output:

```

Langkah: Memasukkan 3
Masukkan: 3, 2, 4, 2, 5
Langkah:
Langkah:
Langkah:
Langkah: Memasukkan 1
Masukkan: 1, 2, 3, 4, 5

```

2.3.5 Hasil MergeSortGUI_2511532002



BAB III

KESIMPULAN

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa algoritma *sorting* merupakan metode yang digunakan untuk mengurutkan data ke dalam urutan tertentu, baik secara menaik (*ascending*) maupun menurun (*descending*). Dalam praktikum ini, algoritma *sorting* berhasil diimplementasikan menggunakan bahasa pemrograman *Java* dengan beberapa metode, yaitu *Shell Sort*, *Quick Sort*, dan *Merge Sort*.

Setiap algoritma memiliki cara kerja yang berbeda dalam melakukan proses pengurutan data. *Shell Sort* bekerja dengan membandingkan elemen berdasarkan jarak tertentu (*gap*) yang semakin diperkecil, *Quick Sort* menggunakan metode *divide and conquer* dengan pemilihan *pivot* untuk membagi data, sedangkan *Merge Sort* mengurutkan data dengan membagi array menjadi beberapa bagian kecil kemudian menggabungkannya kembali dalam keadaan terurut.

Selain itu, praktikum ini juga berhasil mengimplementasikan visualisasi proses *Bubble Sort* dan *Merge Sort* menggunakan *Java GUI (Swing)*, sehingga proses pengurutan data dapat diamati secara bertahap dan lebih mudah dipahami. Melalui praktikum ini, penulis dapat memahami konsep dasar *sorting*, cara kerja masing-masing algoritma, serta penerapan visualisasi algoritma dalam pemrograman *Java*.

3.2 Saran

Saran untuk praktikum selanjutnya adalah agar mahasiswa lebih teliti dalam memahami logika algoritma *sorting* serta langkah-langkah proses pengurutan data pada setiap metode.

DAFTAR PUSTAKA

[1] *GeeksforGeeks*, “Sorting in Java,” 2025. [Online]. Available: <https://www.geeksforgeeks.org/java/sorting-in-java/>. Diakses pada: 20 Mei 2026.

[2] *Ben*, “Bubble Sort, Insertion Sort, and Merge Sort and Big O Notation,” *Medium*, 2023. [Online]. Available: <https://medium.com/@icodewithben/bubble-sort-insertion-sort-and-merge-sort-and-big-o-notation-c98d732cbd75>. Diakses pada: 20 Mei 2026

TUGAS PRAKTIKUM PEKAN 5

1. Kode Program

1.1 Kelas *Lagu_2511532002*

```
package tugasPekan8_2511532002;

public class Lagu_2511532002 {
    String judul_2002;
    String penyanyi_2002;
    int durasi_2002;

    //Konstruktor
    public Lagu_2511532002 (String judul_2002, String penyanyi_2002,
int durasi_2002) {
        this.judul_2002=judul_2002;
        this.penyanyi_2002=penyanyi_2002;
        this.durasi_2002=durasi_2002;
    }
}
```

Penjelasan:

1) Membuat *class*

public class Lagu_2511532002 {

Syntax di atas digunakan untuk membuat *class* bernama *Lagu_2511532002* sebagai *ADT (Abstract Data Type)* untuk menyimpan data mahasiswa.

- 2) Membuat atribut
String judul_2002;
String penyanyi_2002;
int durasi_2002;

Syntax di atas digunakan untuk membuat atribut lagu. Atribut *judul_2002* digunakan untuk menyimpan judul lagu, *penyanyi_2002* digunakan untuk menyimpan nama penyanyi, dan *durasi_2002* digunakan untuk menyimpan durasi lagu dalam satuan detik.

- 3) Membuat konstruktor
public Lagu_2511532002(String judul_2002, String
penyanyi_2002, int durasi_2002) {
 this.judul_2002 = judul_2002;
 this.penyanyi_2002 = penyanyi_2002;
 this.durasi_2002 = durasi_2002;
}

Konstruktor digunakan untuk menginisialisasi data lagu saat objek dibuat, yaitu data judul lagu, nama penyanyi, dan durasi lagu. Dengan konstruktor ini, setiap objek lagu dapat langsung diberikan nilai saat dibuat.

Karena *class Lagu_2511532002* milikmu hanya berisi class, atribut, dan konstruktor, maka penjelasannya cukup sampai 3 *poin* saja. Tidak perlu ada *Getter*, *Setter*, atau *toString()* karena memang tidak ada di programmu.

1.2 Kelas *SortingNIM_2511532002*

```

package tugasPekan8_2511532002;

import java.util.Scanner;

public class SortingNim_2511532002 {
    Lagu_2511532002[] dataLagu_2002=new Lagu_2511532002[20];
    int jumlahData_2002;

    //Methode inputData
    void input_2002() {
        dataLagu_2002[0]=new Lagu_2511532002("Sparks", "Coldplay", 263);
        dataLagu_2002[1]=new Lagu_2511532002("Bonfire", "Wave to earth", 222);
        dataLagu_2002[2]=new Lagu_2511532002("My Love Mine All Mine", "Mitski",
230);
        dataLagu_2002[3]=new Lagu_2511532002("Iris", "The Goo Goo Dolls", 175);
        dataLagu_2002[4]=new Lagu_2511532002("Always", "Daniel Caesar", 270);
        dataLagu_2002[5]=new Lagu_2511532002("Thank You", "Dido", 199);
        dataLagu_2002[6]=new Lagu_2511532002("I Love You, I'm Sorry", "Gracie
Abrams", 225);
        jumlahData_2002=7;
    }

    //Methode ShellSort
    void shellSort_2002() {
        for (int gap_2002=jumlahData_2002/2;
            gap_2002 >0;
            gap_2002 /=2) {

            //Melakukan proses insertion sort berdasarkan gap
            for (int i_2002=gap_2002;
                i_2002 < jumlahData_2002;
                i_2002++) {

                Lagu_2511532002 temp_2002 = ataLagu_2002[i_2002];
                int j_2002;

                //Membandingkan judul Lagu secara Alfabetics
                for (j_2002=i_2002;
                    j_2002 >= gap_2002 && dataLagu_2002[j_2002-
gap_2002].judul_2002.compareToIgnoreCase(temp_2002.judul_2002)>0;
                    j_2002 -=gap_2002) {

                    dataLagu_2002[j_2002]=dataLagu_2002[j_2002-gap_2002];
                }
                dataLagu_2002[j_2002]=temp_2002;
            }
        }
    }

    //Methode QuickSort
    int partition_2002(int low_2002, int high_2002) {
        int pivot_2002 =
            dataLagu_2002[high_2002].durasi_2002;

        int i_2002 = low_2002 - 1;

        for (int j_2002 = low_2002;
            j_2002 < high_2002;
            j_2002++) {

            if (dataLagu_2002[j_2002].durasi_2002 < pivot_2002) {

                i_2002++;

                Lagu_2511532002 temp_2002 =
                    dataLagu_2002[i_2002];
                dataLagu_2002[i_2002] =

```

```

        dataLagu_2002[j_2002];
        dataLagu_2002[j_2002] = temp_2002;
    }
}

Lagu_2511532002 temp_2002 =
    dataLagu_2002[i_2002 + 1];
dataLagu_2002[i_2002 + 1] =
    dataLagu_2002[high_2002];
dataLagu_2002[high_2002] = temp_2002;

return i_2002 + 1;
}

void quickSort_2002(int low_2002, int high_2002) {
    if (low_2002 < high_2002) {
        int pi_2002 =
            partition_2002(low_2002, high_2002);
        quickSort_2002(low_2002, pi_2002 - 1);
        quickSort_2002(pi_2002 + 1, high_2002);
    }
}

//Methode Merge Sort
void merge_2002(int l_2002,
    int m_2002,
    int r_2002) {
    int n1_2002 = m_2002 - l_2002 + 1;
    int n2_2002 = r_2002 - m_2002;

    Lagu_2511532002[] L_2002 = new Lagu_2511532002[n1_2002];
    Lagu_2511532002[] R_2002 = new Lagu_2511532002[n2_2002];

    for (int i_2002 = 0; i_2002 < n1_2002; i_2002++)
        L_2002[i_2002] = dataLagu_2002[l_2002 + i_2002];

    for (int j_2002 = 0; j_2002 < n2_2002; j_2002++)
        R_2002[j_2002] = dataLagu_2002[m_2002 + 1 + j_2002];

    int i_2002 = 0;
    int j_2002 = 0;
    int k_2002 = l_2002;

    while (i_2002 < n1_2002 && j_2002 < n2_2002) {
        if (L_2002[i_2002].judul_2002.compareToIgnoreCase(
            R_2002[j_2002].judul_2002) <= 0) {
            dataLagu_2002[k_2002] = L_2002[i_2002];
            i_2002++;
        } else {
            dataLagu_2002[k_2002] = R_2002[j_2002];
            j_2002++;
        }
        k_2002++;
    }

    while (i_2002 < n1_2002) {
        dataLagu_2002[k_2002] = L_2002[i_2002];
        i_2002++;
        k_2002++;
    }

    while (j_2002 < n2_2002) {
        dataLagu_2002[k_2002] = R_2002[j_2002];
        j_2002++;
        k_2002++;
    }
}

```

```

    }
}

void mergeSort_2002(int l_2002, int r_2002) {
    if (l_2002 < r_2002) {
        int m_2002 = (l_2002 + r_2002) / 2;

        mergeSort_2002(l_2002, m_2002);
        mergeSort_2002(m_2002 + 1, r_2002);

        merge_2002(l_2002, m_2002, r_2002);
    }
}

// Metode Tampilin data
void tampil_2002() {
    for (int i_2002 = 0;
        i_2002 < jumlahData_2002;
        i_2002++) {
        System.out.println(
            (i_2002+1)+" -
"+dataLagu_2002[i_2002].judul_2002+" -
"+dataLagu_2002[i_2002].penyanyi_2002+" -
"+dataLagu_2002[i_2002].durasi_2002+" detik");
    }
}

//Main Program
public static void main(String[] args) {
    Scanner input_2002 = new Scanner(System.in);

    SortingNim_2511532002 playlist_2002 =
        new SortingNim_2511532002();

    playlist_2002.input_2002();

    System.out.println("=== SORTING PLAYLIST NIM 2511532002 ===");
    System.out.println("1. Shell Sort");
    System.out.println("2. Quick Sort");
    System.out.println("3. Merge Sort");

    System.out.print("\nPilih Algoritma Sorting : ");
    int pilih_2002 = input_2002.nextInt();

    System.out.println("\n=== Data Sebelum Sorting ===");
    playlist_2002.tampil_2002();

    switch (pilih_2002) {
        case 1:
            playlist_2002.shellSort_2002();
            System.out.println("\n=== Data Setelah Shell Sort (A-Z)
===");
            break;

        case 2:
            playlist_2002.quickSort_2002(
                0,
                playlist_2002.jumlahData_2002 - 1);
            System.out.println("\n=== Data Setelah Quick Sort
(Durasi) ===");
            break;

        case 3:
            playlist_2002.mergeSort_2002(
                0,
                playlist_2002.jumlahData_2002 - 1);
    }
}

```

```

        System.out.println("\n=== Data Setelah Merge Sort (A-Z)
        ===");
        break;

        default:
            System.out.println("Pilihan tidak valid!");
            return;
    }

    playlist_2002.tampil_2002();
    input_2002.close();
}
}

```

Penjelasan:

1) Method Input Data

```

void input_2002() {
    dataLagu_2002[0]=new Lagu_2511532002("Sparks", "Coldplay", 263);
    ...
    jumlahData_2002=7;
}

```

Method input_2002() digunakan untuk memasukkan data playlist lagu ke dalam array *dataLagu_2002*. Setiap objek lagu berisi judul lagu, nama penyanyi, dan durasi lagu. Variabel *jumlahData_2002* digunakan untuk menyimpan jumlah data lagu yang akan diproses.

2) Method Shell Sort

```

void shellSort_2002() {
    for (int gap_2002=jumlahData_2002/2;
        gap_2002 >0;
        gap_2002 /=2) {

        for (int i_2002=gap_2002;
            i_2002 < jumlahData_2002;
            i_2002++) {

            Lagu_2511532002 temp_2002 = dataLagu_2002[i_2002];
            int j_2002;

            for (j_2002=i_2002;
                j_2002 >= gap_2002 &&
                dataLagu_2002[j_2002-gap_2002].judul_2002
                .compareToIgnoreCase(temp_2002.judul_2002)>0;
                j_2002 -=gap_2002) {

                dataLagu_2002[j_2002]=dataLagu_2002[j_2002-gap_2002];
            }

            dataLagu_2002[j_2002]=temp_2002;
        }
    }
}

```

```

    }
  }
}

```

Shell Sort digunakan untuk mengurutkan data playlist berdasarkan judul lagu secara *ascending (A-Z)*. Algoritma ini bekerja dengan membagi data menggunakan nilai *gap*, kemudian melakukan proses yang mirip dengan *Insertion Sort* pada setiap kelompok data. Nilai *gap* akan terus diperkecil hingga bernilai 1 sehingga seluruh data menjadi terurut.

3) *Method Quick Sort*

```

int partition_2002(int low_2002, int high_2002) {

    int pivot_2002 = dataLagu_2002[high_2002].durasi_2002;

    int i_2002 = low_2002 - 1;

    for (int j_2002 = low_2002;
        j_2002 < high_2002;
        j_2002++) {

        if (dataLagu_2002[j_2002].durasi_2002 < pivot_2002) {

            i_2002++;

            Lagu_2511532002 temp_2002 =
                dataLagu_2002[i_2002];
            dataLagu_2002[i_2002] =
                dataLagu_2002[j_2002];
            dataLagu_2002[j_2002] = temp_2002;
        }
    }

    Lagu_2511532002 temp_2002 =
        dataLagu_2002[i_2002 + 1];
    dataLagu_2002[i_2002 + 1] =
        dataLagu_2002[high_2002];
    dataLagu_2002[high_2002] = temp_2002;

    return i_2002 + 1;
}

void quickSort_2002(int low_2002, int high_2002) {

    if (low_2002 < high_2002) {

        int pi_2002 = partition_2002(low_2002, high_2002);
    }
}

```



```

        quickSort_2002(low_2002, pi_2002 - 1);
        quickSort_2002(pi_2002 + 1, high_2002);
    }
}

```

Method partition_2002() digunakan untuk menentukan posisi pivot pada proses *Quick Sort*. *Pivot* yang digunakan adalah durasi lagu, sehingga lagu dengan durasi lebih kecil akan ditempatkan di sebelah kiri *pivot* dan durasi lebih besar di sebelah kanan *pivot*. Sedangkan, *Quick Sort* digunakan untuk mengurutkan data *playlist* berdasarkan durasi lagu secara *ascending*. Algoritma ini bekerja dengan membagi data menjadi beberapa bagian berdasarkan *pivot*, kemudian mengurutkannya secara rekursif hingga seluruh data terurut.

4) *Methode MergeSort*

```

void merge_2002(int l_2002,
               int m_2002,
               int r_2002) {

    int n1_2002 = m_2002 - l_2002 + 1;
    int n2_2002 = r_2002 - m_2002;

    Lagu_2511532002[] L_2002 = new Lagu_2511532002[n1_2002];
    Lagu_2511532002[] R_2002 = new Lagu_2511532002[n2_2002];

    for (int i_2002 = 0; i_2002 < n1_2002; i_2002++)
        L_2002[i_2002] = dataLagu_2002[l_2002 + i_2002];

    for (int j_2002 = 0; j_2002 < n2_2002; j_2002++)
        R_2002[j_2002] = dataLagu_2002[m_2002 + 1 + j_2002];

    int i_2002 = 0;
    int j_2002 = 0;
    int k_2002 = l_2002;

    while (i_2002 < n1_2002 && j_2002 < n2_2002) {

        if
        (L_2002[i_2002].judul_2002.compareToIgnoreCase(
            R_2002[j_2002].judul_2002) <= 0) {

            dataLagu_2002[k_2002] = L_2002[i_2002];
            i_2002++;
        } else {

            dataLagu_2002[k_2002] = R_2002[j_2002];
            j_2002++;
        }
        k_2002++;
    }
}

```

```

while (i_2002 < n1_2002) {
    dataLagu_2002[k_2002] = L_2002[i_2002];
    i_2002++;
    k_2002++;
}
while (j_2002 < n2_2002) {
    dataLagu_2002[k_2002] = R_2002[j_2002];
    j_2002++;
    k_2002++;
}
}
void mergeSort_2002(int l_2002, int r_2002) {

    if (l_2002 < r_2002) {

        int m_2002 = (l_2002 + r_2002) / 2;

        mergeSort_2002(l_2002, m_2002);
        mergeSort_2002(m_2002 + 1, r_2002);

        merge_2002(l_2002, m_2002, r_2002);
    }
}

```

Method merge_2002() digunakan untuk menggabungkan dua bagian *array* yang telah terurut menjadi satu *array* yang tetap terurut berdasarkan judul lagu secara alfabetis (A-Z).

Merge Sort digunakan untuk mengurutkan data playlist berdasarkan judul lagu secara *ascending* (A-Z). Algoritma ini bekerja dengan membagi data menjadi beberapa bagian kecil, kemudian menggabungkannya kembali dalam keadaan terurut.

5) *Method Tampil Data*

```

void tampil_2002() {
    for (int i_2002 = 0;
        i_2002 < jumlahData_2002;
        i_2002++) {

        System.out.println(
            (i_2002+1)+". "+
            dataLagu_2002[i_2002].judul_2002+" - "+
            dataLagu_2002[i_2002].penyanyi_2002+" - "+
            dataLagu_2002[i_2002].durasi_2002+" detik");
    }
}

```

Method *tampil_2002()* digunakan untuk menampilkan seluruh data playlist lagu yang tersimpan dalam *array*, meliputi judul lagu, nama penyanyi, dan durasi lagu.

6) *Method Main*

```
public static void main(String[] args) {
    Scanner input_2002 = new Scanner(System.in);

    SortingNim_2511532002 playlist_2002 = new SortingNim_2511532002();

    playlist_2002.input_2002();

    System.out.println("=== SORTING PLAYLIST NIM 2511532002 ===");
    System.out.println("1. Shell Sort");
    System.out.println("2. Quick Sort");
    System.out.println("3. Merge Sort");

    System.out.print("\nPilih Algoritma Sorting : ");
    int pilih_2002 = input_2002.nextInt();

    System.out.println("\n=== Data Sebelum Sorting ===");
    playlist_2002.tampil_2002();

    switch (pilih_2002) {

        case 1:
            playlist_2002.shellSort_2002();
            System.out.println("\n=== Data Setelah Shell Sort (A-Z) ===");
            break;

        case 2:
            playlist_2002.quickSort_2002(0,
            playlist_2002.jumlahData_2002 - 1);
            System.out.println("\n=== Data Setelah Quick Sort (Durasi) ===");
            break;

        case 3:
            playlist_2002.mergeSort_2002(0,
            playlist_2002.jumlahData_2002 - 1);
            System.out.println("\n=== Data Setelah Merge Sort (A-Z) ===");
            break;

        default:
            System.out.println("Pilihan tidak valid!");
            return;
    }
}
```

```

        playlist_2002.tampil_2002();
        input_2002.close();
    }
}

```

Method *main()* digunakan sebagai method utama untuk menjalankan program. Method ini berfungsi untuk menampilkan menu pilihan algoritma *sorting*, menerima input pengguna, menampilkan data sebelum *sorting*, menjalankan algoritma yang dipilih (*Shell Sort*, *Quick Sort*, atau *Merge Sort*), dan menampilkan hasil pengurutan data playlist lagu.

Method utama yang digunakan untuk menjalankan program dan menampilkan GUI aplikasi.

2. Hasil

```

174 playlist_2002.input_2002();
175
176
177 System.out.println("==== SORTING PLAYLIST NIM 2511532002 ====");
178 System.out.println("1. Shell Sort");
179 System.out.println("2. Quick Sort");
180 System.out.println("3. Merge Sort");
181
182 System.out.print("Pilih Algoritma Sorting : ");
183 int pilih_2002 = input_2002.nextInt();
184
185 System.out.println("==== Data Sebelum Sorting ====");
186 playlist_2002.tampil_2002();
187
188 switch (pilih_2002) {
189     case 1:
190         playlist_2002.shellSort_2002();
191         System.out.println("==== Data Setelah Shell Sort (A-Z) ====");
192         break;
193     case 2:
194         playlist_2002.quickSort_2002();
195         System.out.println("==== Data Setelah Quick Sort (Durasi) ====");
196         break;
197     case 3:
198         playlist_2002.mergeSort_2002();
199         System.out.println("==== Data Setelah Merge Sort (A-Z) ====");
200         break;
201     default:
202         System.out.println("Pilihan tidak valid!");
203 }
204
205 playlist_2002.tampil_2002();
206 input_2002.close();
207 }

```

Console Output:

```

===== SORTING PLAYLIST NIM 2511532002 =====
1. Shell Sort
2. Quick Sort
3. Merge Sort

Pilih Algoritma Sorting : 2

==== Data Sebelum Sorting ====
1. Sparks - Coldplay - 263 detik
2. Bonfire - Move to earth - 222 detik
3. My Love Mine All Mine - Mitski - 230 detik
4. Iris - The Goo Goo Dolls - 175 detik
5. Always - Daniel Caesar - 270 detik
6. Thank You - Dido - 199 detik
7. I Love You, I'm Sorry - Gracie Abrams - 225 detik

==== Data Setelah Quick Sort (Durasi) ====
1. Iris - The Goo Goo Dolls - 175 detik
2. Thank You - Dido - 199 detik
3. Bonfire - Move to earth - 222 detik
4. I Love You, I'm Sorry - Gracie Abrams - 225 detik
5. My Love Mine All Mine - Mitski - 230 detik
6. Sparks - Coldplay - 263 detik
7. Always - Daniel Caesar - 270 detik

```

Quick Sort dipilih untuk mengurutkan data lagu berdasarkan durasi lagu secara *ascending* (terpendek ke terpanjang). Algoritma ini memiliki performa yang sangat baik dengan kompleksitas rata-rata merupakan salah satu algoritma *sorting* tercepat dengan kompleksitas rata-rata $O(n \log n)$ sehingga mampu mengurutkan data dengan cepat. Dalam program playlist, pengurutan berdasarkan durasi membantu pengguna melihat lagu yang paling singkat hingga paling panjang secara terstruktur.

3. Kesimpulan

Berdasarkan penugasan yang telah dilakukan, dapat disimpulkan bahwa algoritma *Shell Sort*, *Quick Sort*, dan *Merge Sort* berhasil diimplementasikan pada program playlist lagu menggunakan bahasa pemrograman *Java*. Setiap algoritma memiliki cara kerja yang berbeda dalam mengurutkan data, baik berdasarkan judul lagu maupun durasi lagu.

Program yang dibuat juga memungkinkan pengguna memilih algoritma sorting yang akan digunakan melalui menu pilihan. Melalui penugasan ini, penulis dapat memahami konsep dasar *sorting*, implementasi berbagai algoritma *sorting*, serta perbedaan cara kerja dan efisiensi masing-masing algoritma dalam pengolahan data playlist lagu.